

API 参考

用 libterae 构建 MQTT 服务器或客户端

Emilio Schininà，自动化工程硕士
版本 0.2
2026 年 11 月 8 日

版权所有 © 2026 Emilio Schininà, 自动化工程硕士。libterae 以 GNU 通用公共许可证第 3 版或任何更高版本, 或以商业许可证发布。

目录

1 入门 **1**

1.1 两个接口,两种运行方式 1
1.2 每次调用都适用的规则 2
1.3 构建程序 3
1.4 状态码 4
1.5 函数的展示方式 5

2 服务器 **7**

2.1 概览 7
2.2 服务器的生命周期 9
2.3 配置 9
2.4 内存 13
2.5 初始化 15
2.5.1 `mqtt_server_config_default` 15
2.5.2 `mqtt_server_meta_words` 16
2.5.3 `mqtt_server_init` 16
2.6 用 `platform/` 运行 17
2.6.1 `mqtt_net_server_run` 17
2.6.2 `mqtt_net_server_counters` 18
2.7 在程序自己的循环中运行 19
2.7.1 `mqtt_server_connection_open` 20
2.7.2 `mqtt_server_connection_closed` 21
2.7.3 `mqtt_server_receive_wanted` 21
2.7.4 `mqtt_server_receive_capacity` 22
2.7.5 `mqtt_server_receive_space` 23
2.7.6 `mqtt_server_receive_commit` 24

api-reference-zh.tex Page iv November 8, 2026 13:47

2.7.7 mqtt_server_receive . . . 25
2.7.8 mqtt_server_transmit_view . . . 26
2.7.9 mqtt_server_transmit_consume . . . 27
2.7.10 mqtt_server_should_close . . . 27
2.7.11 mqtt_server_next_pending . . . 28
2.7.12 mqtt_server_tick . . . 28
2.7.13 mqtt_server_take_activity . . . 29
2.7.14 mqtt_server_accepted . . . 29
2.7.15 mqtt_server_keep_alive . . . 30
2.7.16 mqtt_server_idle . . . 30
2.7.17 mqtt_server_disconnect . . . 31
2.8 来自程序的消息 . . . 32
2.8.1 mqtt_server_publish . . . 32
2.8.2 mqtt_server_retained_next . . . 33
2.8.3 mqtt_server_retained_restore . . . 34
2.9 让会话状态在重启后仍在 . . . 34
2.9.1 mqtt_net_server_store_config . . . 35
2.9.2 mqtt_net_server_restore . . . 36
2.10 在程序自己的循环中使用存储区 . . . 36
2.10.1 mqtt_server_restore . . . 38
2.10.2 mqtt_server_store_commit . . . 39
2.10.3 mqtt_server_stored . . . 40
2.10.4 mqtt_server_store_failed . . . 40
2.10.5 mqtt_server_store_open . . . 41
2.10.6 mqtt_server_snapshot_due . . . 41
2.10.7 mqtt_server_snapshot_begin . . . 42
2.10.8 mqtt_server_snapshot_next . . . 43
2.10.9 mqtt_server_snapshot_done . . . 44
2.10.10 mqtt_server_snapshot_abort . . . 44
2.11 检查运行中的服务器 . . . 44
2.11.1 mqtt_server_pool_audit . . . 45

3 客户端 47
3.1 概览 . . . 47
3.2 客户端的生命周期 . . . 49
3.3 内存 . . . 49
3.4 事件 . . . 50

api-reference-zh.tex Page v November 8, 2026 13:47

3.5 初始化 52
3.5.1 mqtt_client_init 52
3.5.2 mqtt_client_assume_session 53
3.6 连接 54
3.6.1 mqtt_client_connect 54
3.6.2 mqtt_client_auth 55
3.6.3 mqtt_client_disconnect 56
3.6.4 mqtt_client_should_close 56
3.6.5 mqtt_client_connection_lost 57
3.7 消息和订阅 58
3.7.1 mqtt_client_publish 58
3.7.2 mqtt_client_subscribe 60
3.7.3 mqtt_client_unsubscribe 61
3.8 用 platform/ 运行 62
3.8.1 mqtt_net_client_open 62
3.8.2 mqtt_net_client_step 63
3.8.3 mqtt_net_client_close 64
3.9 在程序自己的循环中运行 64
3.9.1 mqtt_client_receive_capacity 65
3.9.2 mqtt_client_receive_space 65
3.9.3 mqtt_client_receive_commit 66
3.9.4 mqtt_client_receive 67
3.9.5 mqtt_client_transmit_view 68
3.9.6 mqtt_client_transmit_consume 68
3.9.7 mqtt_client_tick 69
3.10 让会话状态在重启后仍在 69
3.10.1 mqtt_client_store_init 71
3.10.2 mqtt_net_client_store_config 72
3.10.3 mqtt_net_client_restore 73
3.10.4 mqtt_client_store_identifier 74
3.11 在程序自己的循环中使用存储区 74
3.11.1 mqtt_client_restore 75
3.11.2 mqtt_client_store_commit 76
3.11.3 mqtt_client_stored 77
3.11.4 mqtt_client_store_failed 77
3.11.5 mqtt_client_store_open 78

api-reference-zh.tex Page vi November 8, 2026 13:47

3.11.6 mqtt_client_snapshot_due . . . 78
3.11.7 mqtt_client_snapshot_begin . . . 79
3.11.8 mqtt_client_snapshot_next . . . 80
3.11.9 mqtt_client_snapshot_done . . . 80
3.11.10 mqtt_client_snapshot_abort . . . 81

4 两者共用 83
4.1 字节视图 . . . 83
4.1.1 mqtt_utf8_from_text . . . 84
4.2 属性 . . . 84
4.2.1 mqtt5_properties_clear . . . 86
4.2.2 mqtt5_properties_mark . . . 87
4.2.3 mqtt5_properties_has . . . 87
4.2.4 mqtt5_property_next . . . 88
4.3 原因码 . . . 88
4.3.1 mqtt_reason_clear . . . 88
4.3.2 mqtt_connect_clear . . . 89
4.3.3 mqtt5_reason_code . . . 89
4.4 TLS . . . 89
4.4.1 mqtt_tls_available . . . 90
4.5 存储区 . . . 90
4.5.1 mqtt_store_fault_name . . . 93
4.5.2 mqtt_platform_store_open . . . 94
4.5.3 mqtt_platform_store_close . . . 94
4.6 系统 . . . 96
4.6.1 mqtt_platform_time_ms . . . 96
4.6.2 mqtt_platform_random_bytes . . . 97
4.6.3 mqtt_platform_stop_requested . . . 97
4.6.4 mqtt_platform_request_stop . . . 98
4.7 库的版本 . . . 100
4.7.1 mqtt_library_compatible . . . 100
4.7.2 mqtt_library_version . . . 101
4.7.3 mqtt_library_page_size . . . 101

5 示例 103
5.1 服务器 . . . 103
5.2 客户端 . . . 110

api-reference-zh.tex Page vii November 8, 2026 13:47

A 全部函数 117

1

入门

本文档列出程序用 libterae 实现 MQTT 服务器或 MQTT 客户端时调用的每个函数、每个函数接收什么和交回什么，以及程序为它们填写的结构。服务器在第 2 章，客户端在第 3 章，所以只实现其中一个的程序只需读它那一章，再加上讲两者共用部分的第 4 章。第 5 章是一些原样就能编译、运行的程序。

这些函数按 1.0.5 版的样子描述。每个函数的头文件在它自己的注释里有同样的描述，两者不一致时以头文件为准。

1.1 两个接口，两种运行方式

服务器是一个 `mqtt_server`，由 `base_server.h` 声明；客户端是一个 `mqtt_client`，来自 `base_client.h`。两者自身都不做任何输入输出。它们接收到达的字节，交回要发送的字节，并由调用者告知时间，如图 1.1 所示。在它们和网络之间搬运字节的是一个循环，程序有两种可选：

- **`platform/` 的循环。** `mqtt_net_server_run`（§2.6.1）运行整个服务器，`mqtt_net_client_step`（§3.8.2）每次把客户端的字节搬运一步。它们负责套接字、TLS、服务器的 WebSocket 以及等待，适用于库有对应文件夹的那些系统。这是捷径，示例走的就是这条路。
- **程序自己的循环。** 程序读写自己的套接字，或者无线电、串口，并把字节交给服务器或客户端。用 `OS=none` 构建的库只有这一种方式。第 2.7 节和第 3.9 节给出要调用的函数。

表 1.1 列出程序为两者各自包含哪些头文件。

api-reference-zh.tex Page 2 November 8, 2026 13:47

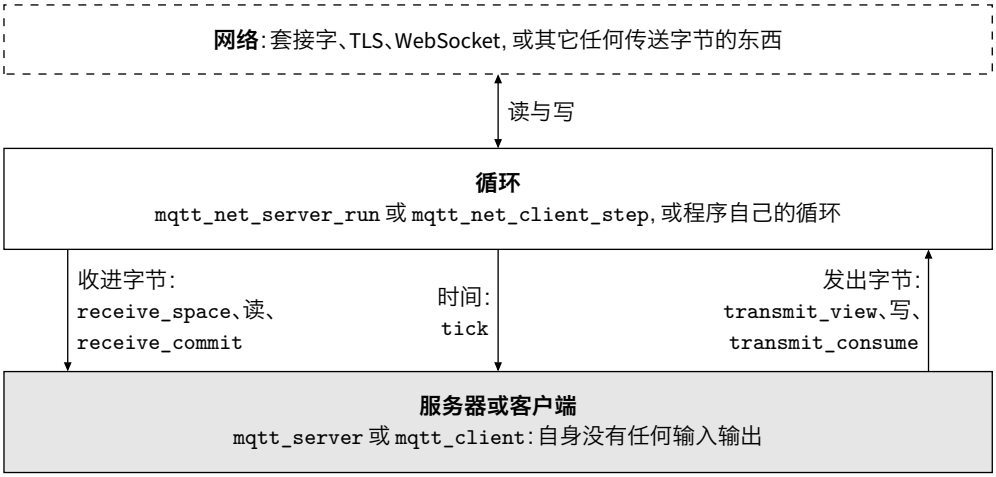


图 1.1 字节和时间怎样到达服务器或客户端。调用的名字以 mqtt_server_ 或 mqtt_client_ 开头

表 1.1 程序包含的头文件

用于	服务器	客户端
接口	base_server.h	base_client.h
platform/ 的循环	base_net.h	base_net.h
时钟、随机字节、停止	base_platform.h	base_platform.h
保存会话状态	base_store.h；platform/ 的存储区另需 base_platform.h	同左
原因码	v5_rc.h、v311_rc.h、v31_rc.h	同左
属性标识符	v5_prop.h	v5_prop.h
订阅选项	–	v5_subscribe.h

1.2 每次调用都适用的规则

- **内存全部由程序提供。**服务器或客户端在初始化时，就通过一个 mqtt_server_storage 或 mqtt_client_storage 得到它将用到的每一个数组，此后从不分配内存。这些数组可以是静态的，可以在一直存在的栈上，也可以在其它任何地方，但必须与服务器或客户端存在得一样久。它们不必事先清零。它们的大小就是上限：任何

api-reference-zh.tex Page 3 November 8, 2026 13:47

东西用完都是一种正常的结果，按协议允许的方式报告，绝不会导致崩溃。

- **把对象当作不透明的。**`mqtt_server` 和 `mqtt_client` 出现在头文件里，只是为了让程序能为它们提供内存。请用函数，不要用字段。
- **同一时间只有一个线程。**没有任何加锁。对同一个服务器或同一个客户端的调用必须一个接一个地进行。不同的服务器和客户端之间没有任何共享，可以在不同的线程中使用。
- **时间是一个参数。**需要时间的调用以 `now_ms` 接收它，单位为毫秒，取自一个只会向前走的时钟，例如 `mqtt_platform_time_ms`(§4.6.1)。协议规定的间隔保留协议的单位，即秒，Keep Alive 就是如此。
- **字符串是视图。**`mqtt_utf8`、`mqtt_binary` 和 `mqtt_payload` 保存一个长度和一个指向字节的指针，这些字节不归它们所有，末尾也没有 NUL。调用收到的内容如果要保留，会被复制；事件交出的内容只在事件期间有效(第 4.1 节)。
- **结果是 `mqtt_status`。**可能失败的函数返回表 1.2 中的某个值。`MQTT_OK` 表示它完成了所要求的事。

1.3 构建程序

程序针对 `include/` 编译，使用与库构建时相同的页大小，并与 `libterae.a` 链接：

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U server.c build/libterae.a -o server
```

程序交给库的那些数组，大小由 `MQTT_PAGE_SIZE` 决定，而归档文件不记录它，所以两者必须一致。带 TLS 的构建在 `libterae.a` 之后链接 Mbed TLS 的三个归档文件，Windows 程序还要链接 `ws2_32` 和 `bcrypt`。程序也可以从 `make shared` 构建的共享库取得该库，由系统在程序启动时加载；这时程序要检查那个库是否可用 (§4.7)。各种构建见《构建 libterae》。

api-reference-zh.tex Page 4 November 8, 2026 13:47

1.4 状态码

表 1.2 mqtt_status 的取值, 来自 base_status.h

值	含义
MQTT_OK	完成了所要求的事。
MQTT_INCOMPLETE	还没有: 没有完整的一项, 没有可读的, 或者什么也没被取走。
MQTT_MALFORMED_PACKET	字节不符合报文格式, 或者某个字符串不是有效的 UTF-8, 含有它的报文会因此成为畸形报文。
MQTT_PROTOCOL_ERROR	报文格式正确, 但违反了协议。
MQTT_UNSUPPORTED_PROTOCOL_VERSION	CONNECT 要求的版本是库所没有的。
MQTT_CLIENT_IDENTIFIER_NOT_VALID	服务器不接受这个客户端标识符。
MQTT_BAD_USER_NAME_OR_PASSWORD	用户名或密码被拒绝。
MQTT_NOT_AUTHORIZED	被授权检查拒绝。
MQTT_BAD_AUTHENTICATION_METHOD	服务器不支持这个认证方法。
MQTT_TOPIC_NAME_INVALID	主题名无效。
MQTT_TOPIC_FILTER_INVALID	主题过滤器无效。
MQTT_TOPIC_ALIAS_INVALID	主题别名无效。
MQTT_PACKET_TOO_LARGE	报文超过了它的上限。
MQTT_RECEIVE_MAXIMUM_EXCEEDED	在途的 QoS 1 和 QoS 2 消息多于 Receive Maximum 所允许的。
MQTT_PACKET_IDENTIFIER_IN_USE	该报文标识符正在使用。
MQTT_PACKET_IDENTIFIER_NOT_FOUND	该报文标识符没有在使用。
MQTT_PAYLOAD_FORMAT_INVALID	有效载荷与它的 Payload Format Indicator 不符。
MQTT_BUFFER_TOO_SMALL	结果放不进它该去的地方。
MQTT_QUOTA_EXCEEDED	程序提供的某个数组已满。
MQTT_INVALID_ARGUMENT	某个参数超出范围或为 NULL, 或者某个对象没有初始化。

续页

api-reference-zh.tex Page 5 November 8, 2026 13:47

表 1.2 mqtt_status 的取值(续)

值	含义
MQTT_INVALID_STATE	现在不允许,例如在连接被接受之前。
MQTT_NOT_FOUND	要找的项不存在。
MQTT_CONNECTION_CLOSED	网络连接已经结束,或者无法建立。
MQTT_TLS_ERROR	TLS 失败,或者库构建时没有带 TLS。
MQTT_RING_FULL	没有足够的空间容纳全部字节,所以一个也没有加入。
MQTT_STORE_DAMAGED	保存的状态已损坏,或者放不下,所以其中什么也没有被采用(第 4.5 节)。

1.5 函数的展示方式

每个函数有自己的一小节:一句话说明它做什么;它的声明,与头文件里的一样,框的底部写出头文件的名字;以及一张表,列出它的参数和返回值。参数名下面是它的方向:“输入”表示函数读取的,“输出”表示函数写入的,“输入输出”表示两者都有。对指针而言,方向指的是它所指向的东西。每个指针都必须有效,或者在它那一行说可以时为 NULL。在其他地方传入的 NULL 指针,会像没有初始化的对象一样被拒绝:调用不改变任何东西,返回 MQTT_INVALID_ARGUMENT、0、false,或者它在没有找到时给出的值。

```
uint32_t mqtt_server_tick(mqtt_server *server, uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
now_ms 输入	时间,以毫秒计。
返回	距离应当再次调用它还有多少毫秒。

2

服务器

服务器在同一个监听套接字上接受 MQTT 3.1、3.1.1 和 5.0 客户端的网络连接，并完成协议对服务器的全部要求：会话、订阅、保留消息、遗嘱消息、QoS 1 和 2、主题别名、共享订阅以及增强认证。程序只决定它有多少内存、它的配置，以及谁可以连接、发布和订阅。

2.1 概览

表 2.1 服务器的函数

函数	作用	§
初始化		
`mqtt_server_config_default`	用默认值填写一份配置	2.5.1
`mqtt_server_meta_words`	计算存储所需的页元数据	2.5.2
`mqtt_server_init`	初始化服务器	2.5.3
用 `platform/` 运行		
`mqtt_net_server_run`	运行服务器，直到程序停止	2.6.1
`mqtt_net_server_counters`	读取网络层的计数	2.6.2
在程序自己的循环中运行		
`mqtt_server_connection_open`	报告一个新连接	2.7.1
`mqtt_server_connection_closed`	报告一个连接已经断开	2.7.2
`mqtt_server_receive_wanted`	说明现在是否读取某个连接	2.7.3
`mqtt_server_receive_capacity`	说明可以交给它多少字节	2.7.4

续页

api-reference-zh.tex Page 8 November 8, 2026 13:47

表 2.1 服务器的函数(续)

函数	作用	§
`mqtt_server_receive_space`	给出读入字节的位置	2.7.5
`mqtt_server_receive_commit`	接收读入其中的字节	2.7.6
`mqtt_server_receive`	从程序的缓冲区接收字节	2.7.7
`mqtt_server_transmit_view`	给出要发送的字节	2.7.8
`mqtt_server_transmit_consume`	说明其中有多少已经发出	2.7.9
`mqtt_server_should_close`	说明何时关闭套接字	2.7.10
`mqtt_server_next_pending`	交出有事要做的连接	2.7.11
`mqtt_server_tick`	运行定时器	2.7.12
`mqtt_server_take_activity`	说明一个连接发来了什么	2.7.13
`mqtt_server_accepted`	说明它的 CONNECT 是否已被接受	2.7.14
`mqtt_server_keep_alive`	给出正在生效的 Keep Alive	2.7.15
`mqtt_server_idle`	说明一个连接是否什么也没持有	2.7.16
`mqtt_server_disconnect`	从服务器一侧关闭连接	2.7.17
来自程序的消息		
`mqtt_server_publish`	从服务器一侧发布消息	2.8.1
`mqtt_server_retained_next`	交出保留消息, 以便保存	2.8.2
`mqtt_server_retained_restore`	放回一条保存过的保留消息	2.8.3
保存会话状态		
`mqtt_net_server_store_config`	设置配置, 以使用 `platform/` 的存储区	2.9.1
`mqtt_net_server_restore`	放回那个存储区保存的东西	2.9.2
`mqtt_server_restore`	放回存储区保存的东西, 介质不限	2.10.1
`mqtt_server_store_commit`	把一轮的记录封成一组	2.10.2
`mqtt_server_stored`	说明各组已经存储, 它们的应答可以发出	2.10.3
`mqtt_server_store_failed`	说明记录可能已经丢失	2.10.4
`mqtt_server_store_open`	说明是否有记录等待封组	2.10.5
`mqtt_server_snapshot_due`	说明是否应该开始一次快照	2.10.6

续页

api-reference-zh.tex Page 9 November 8, 2026 13:47

表 2.1 服务器的函数(续)

函数	作用	§
`mqtt_server_snapshot_begin`	开始一次快照	2.10.7
`mqtt_server_snapshot_next`	给出它的下一条记录	2.10.8
`mqtt_server_snapshot_done`	说明它在每份副本中都已完整	2.10.9
`mqtt_server_snapshot_abort`	放弃它	2.10.10
检查		
`mqtt_server_pool_audit`	检查块池	2.11.1

2.2 服务器的生命周期

1. 用 `mqtt_server_config_default`（§2.5.1）填写一个 `mqtt_server_config`，再改动程序需要的部分：它的各种上限，以及决定谁可以连接、发布和订阅的回调（第 2.3 节）。
2. 要让会话在重启后仍在，就打开一个存储区，并设置配置，把会话保存在那里（第 2.9 节）。
3. 在一个 `mqtt_server_storage` 中提供内存（第 2.4 节）。
4. 用 `mqtt_server_init`（§2.5.3）初始化服务器；如果有存储区，就放回它保存的东西。
5. 运行它：用 `mqtt_net_server_run`（§2.6.1），它在程序被要求停止时返回；或者在程序自己的循环中运行（第 2.7 节）。

2.3 配置

`mqtt_server_config_default`（§2.5.1）写入表 2.2 的每个字段。MQTT 5.0 也有同名属性的上限，含义与那个属性相同；与协议自身默认值不同的那些，服务器会告诉 5.0 的客户端。

api-reference-zh.tex Page 10 November 8, 2026 13:47

表 2.2 `mqtt_server_config` 的字段

字段	默认值	含义
回调，各自为 `NULL` 表示没有		
`user_tag`	0	交给每个回调（也包括 `store_append`），用来找到程序自己的数据。`mqtt_net_server_store_config`（§2.9.1）把它设为存储区。
`authenticate`	`NULL`：均可连接	决定一个客户端是否可以连接。
`auth_step`	`NULL`：没有方法	一步一步地进行增强认证。
`authorize_publish`	`NULL`：均可发布	决定一个客户端是否可以向某个主题发布。
`authorize_subscribe`	`NULL`：均可订阅	决定一个客户端是否可以订阅某个主题过滤器。
`random_bytes`	`NULL`：按编号	给出每个分配的客户端标识符所用的随机字节。按编号的标识符可以被猜到，猜中别人标识符的客户端可以接管它的会话，所以请提供一个。
对客户端所发内容的限制		
`maximum_packet_size`	268,435,455	从客户端接收的最大报文。
`maximum_connect_size`	1 MiB	来自尚未被接受的客户端的最大报文。
`receive_maximum`	65,535	一个客户端可以同时进行中的 QoS 2 消息的最大数目，1 到 65,535。
`maximum_client_identifier`	256	最长的客户端标识符，至少 23。
`topic_alias_maximum`	16	客户端可以设置的主题别名数目，它也决定存储中 `aliases` 的大小。
`maximum_qos`	2	Maximum QoS。大于 2 的值按 2 处理。

续页

api-reference-zh.tex Page 11 November 8, 2026 13:47

表 2.2 `mqtt_server_config` 的字段(续)

字段	默认值	含义
对服务器所保存和发送内容的限制		
`maximum_inflight`	1,024	已发给一个客户端、尚未得到确认的 QoS 1 和 2 消息，1 到 65,535。
`maximum_queued`	4,096	等待发给一个会话的消息。
`maximum_session_expiry`	永不	最长的会话过期间隔, 以秒计。
`persistent_session_expiry`	永不	给 MQTT 3.1 和 3.1.1 中 Clean Session 为 0 的会话的会话过期间隔。
时间		
`connect_timeout_ms`	30,000	新连接发送它的 CONNECT 并完成认证的时限。0: 不限。
`retry_interval_ms`	30,000	MQTT 3.1 客户端没有得到应答的消息隔多久重发, 每次加倍。0: 只在会话继续时重发。
`server_keep_alive`	0	Server Keep Alive, 以秒计, 0 表示采用客户端的。
特性		
`retain_available`	true	Retain Available。
`wildcard_subscription_available`	true	Wildcard Subscription Available。
`subscription_identifier_available`	true	Subscription Identifier Available。
`shared_subscription_available`	true	Shared Subscription Available。
`shared_subscription_311`	false	在 MQTT 3.1.1 中也把 `\$share/` 当作共享订阅, 与其它服务器的做法一样。
`dollar_publish_allowed`	false	允许 3.1.1 和 5.0 的客户端向以 $ 开头的主题发布。

续页

api-reference-zh.tex Page 12 November 8, 2026 13:47

表 2.2 `mqtt_server_config` 的字段(续)

字段	默认值	含义
`report_no_matching_subscribers`	false	没有人订阅时，以原因码 0x10 回应。
块池，以块计		
`pool_soft_quota`	4	当块池只剩 `pool_reserve` 个空闲块时，一个连接可以持有的数量。
`pool_hard_quota`	16	一个连接最多可以持有的数量。其中约四分之三是从它的环发送的最大报文；更大的报文从页区发送。
`pool_reserve`	64	为回复和刚开始的连接保留的空闲块。
`pool_borrow_reserve`	128	不让超过软配额的连接动用的空闲块，至少为 `pool_reserve`。
存储区		
`store_append`	`NULL`：什么都不保存	把日志的每条记录交给程序保存(第 2.9 节)。
`store_sessions`	`MQTT_STORE_MEMORY`	会话的记录必须经受得住什么：它们的订阅、遗嘱消息和生命期。`MQTT_STORE_MEMORY` 什么都不写。
`store_messages`	`MQTT_STORE_MEMORY`	消息的记录必须经受得住什么：等待中、在途和保留的消息，以及收到的 QoS 2 报文标识符。`MQTT_STORE_MEMORY` 保留会话，但不保留它们的消息。
`store_snapshot_bytes`	64 MiB	日志写到多少字节后应做一次快照，好让日志重新开始。

回调在需要它们的那次调用中运行，在程序的线程上。它们的类型如下：

api-reference-zh.tex Page 13 November 8, 2026 13:47


```
typedef mqtt_status (*mqtt_server_authenticate)(uint32_t user_tag,
                                                const mqtt_connect *packet);
typedef mqtt_status (*mqtt_server_auth_step)(uint32_t user_tag,
                                             uint32_t connection,
                                             const mqtt_utf8 *method,
                                             const mqtt_binary *data_in,
                                             mqtt_binary *data_out);
typedef bool (*mqtt_server_authorize)(uint32_t user_tag,
                                      const mqtt_utf8 *client_identifier,
                                      const mqtt_utf8 *topic);
typedef mqtt_status (*mqtt_server_random_bytes)(uint32_t user_tag,
                                                uint8_t *data,
                                                uint32_t length);
```

见 *include/base_server.h*

- **authenticate**。得到整个 CONNECT 报文，其中包括用户名和密码（表 3.4），返回 MQTT_OK、MQTT_BAD_USER_NAME_OR_PASSWORD 或 MQTT_NOT_AUTHORIZED。
- **auth_step**。得到收到的认证方法和认证数据，把要发送的认证数据写入 data_out，其中的字节在它返回之前必须一直有效。完成时返回 MQTT_OK，要继续时返回 MQTT_INCOMPLETE，否则返回一个失败。连接的索引让它可以为这次交换保存自己的状态。
- **authorize_publish、authorize_subscribe**。得到客户端标识符以及主题名或主题过滤器，客户端被允许时返回 true。
- **random_bytes**。从攻击者无法预测的来源填充 length 个字节，返回 MQTT_OK；没有这样的来源时返回其它任何值。mqtt_platform_random_bytes（§4.6.2）就是这样一个来源。

2.4 内存

一个 mqtt_server_storage 指明服务器将用到的每个数组，以及它有多少项。这些数目就是服务器的容量。第 5.1 节的清单 server_setup.c 填写了一个。

api-reference-zh.tex Page 14 November 8, 2026 13:47

表 2.3 mqtt_server_storage 的字段

字段	应提供
连接	
connections、connection_count	同时打开的每个连接一个 mqtt_connection。
aliases	connection_count 乘以 topic_alias_maximum 项 mqtt_topic_alias；该最大值为 0 时为 NULL。
传输中的字节	
pool_bytes、pool_next	pool_block_count 乘以 pool_block_size 个字节，以及每块一个 uint32_t。每个连接要发送的字节和收到的字节都来自这里。
pool_block_size	一块的字节数，至多 4,096。乘以 pool_hard_quota 后至少为 1,024。
pool_block_count	多于 pool_borrow_reserve 与 pool_hard_quota 之和。
receive_scratch、receive_scratch_size	至少 16 个字节，分几块到达的报文在这里拼合。它的大小就是保存在块中的最大报文：更大的去页区。
会话和消息	
sessions、session_count	保存的每个会话一个 mqtt_session，无论它是否连接着。
session_table、session_table_count	至少 session_count 项，用于按客户端标识符查找会话。
deliveries、delivery_count	发往某一个会话的每条消息一个 mqtt_delivery。
messages、message_count	存储的每条消息一个 mqtt_message，不论它发往多少个会话。
subscriptions、subscription_count	每个会话的每个订阅一个 mqtt_subscription。
groups、group_count	每个共享订阅一个 mqtt_share_group；没有时为 NULL 和 0。
主题	
nodes、node_count	每个被订阅或被保留的主题的每一层一个 mqtt_tree_node，至多 65,535。

续页

api-reference-zh.tex Page 15 November 8, 2026 13:47

表 2.3 mqtt_server_storage 的字段(续)

字段	应提供
branches、branch_count	至少 node_count − 2 项 mqtt_tree_branch。
topic_text、topic_text_pages	若干 MQTT_PAGE_SIZE 字节的页，存放各层的文本。
一切大小不定的东西	
arena_bytes、arena_pages	若干 MQTT_PAGE_SIZE 字节的页，存放消息、客户端标识符和正在拼合的报文。
page_meta、page_meta_words	至少为 mqtt_server_meta_words(§2.5.2)对存储其余部分给出的字数。

在按使用情况映射内存的系统上，从未被触及的页不占任何开销，所以页区可以给得宽裕些。

2.5 初始化

2.5.1 mqtt_server_config_default

用表 2.2 的默认值填写一份配置。每个字段都会被写入，所以这块内存事先可以是任何内容。

```
void mqtt_server_config_default(mqtt_server_config *config);
```

见 *include/base_server.h*

参数	含义
config 输出	配置。
返回	无。

api-reference-zh.tex Page 16 November 8, 2026 13:47

2.5.2 mqtt_server_meta_words

计算存储所需的页元数据的字数：每个页区的每一页 MQTT_PAGE_META_WORDS 个。先填写存储的其它每个字段。在编译时确定 page_meta 大小的程序可以自己计算页数，就像 server_setup.c 那样。

```
uint32_t mqtt_server_meta_words(const mqtt_server_storage *storage);
```

见 *include/base_server.h*

参数	含义
storage 输入	存储，其中的数目和页数都已设好。
返回	page_meta 的字数；没有数组能容纳它们时为 0xFFFFFFFF。

2.5.3 mqtt_server_init

初始化一个服务器。配置会被复制，对内存的描述也会被复制：数组本身从此归服务器所有。

```
mqtt_status mqtt_server_init(mqtt_server *server,
                             const mqtt_server_config *config,
                             const mqtt_server_storage *storage);
```

见 *include/base_server.h*

参数	含义
server 输出	服务器。
config 输入	配置。
storage 输入	内存。
返回	MQTT_OK；配置或存储违反表 2.2 或表 2.3 的某条规则时为 MQTT_INVALID_ARGUMENT。

其它每个函数都拒绝没有经过它初始化的服务器。

api-reference-zh.tex Page 17 November 8, 2026 13:47

2.6 用 platform/ 运行

2.6.1 mqtt_net_server_run

运行一个服务器，直到程序被要求停止。它在选项指定的端口上监听，接受连接，分级等待就绪的套接字，使每次等待只需查看少数几个，搬进搬出它们的字节，处理 TLS 和 WebSocket，并运行定时器，这一切都在调用它的线程上进行。

```
mqtt_status mqtt_net_server_run(mqtt_server *server,
                                mqtt_net_connection *connections,
                                const mqtt_net_server_options *options);
```

见 *include/base_net.h*

参数	含义
server 输入输出	一个已由 mqtt_server_init（§2.5.3）初始化的服务器。
connections 输出	服务器的每个连接一个 mqtt_net_connection：共 connection_count 项。
options 输入	在哪里监听，用什么（表 2.4）。
返回	停止之后为 MQTT_OK，否则为它无法启动的原因。

当 mqtt_platform_stop_requested（§4.6.3）这样说时，它就停止：在 POSIX 系统上，是在收到 SIGINT 或 SIGTERM 之后，它会开始监视这两个信号；在任何系统上，是在 mqtt_platform_request_stop（§4.6.4）之后。然后它关闭每个连接并返回。TLS 端口需要带 TLS 构建的库，证书和密钥由它自己加载。

选项中有存储区时，它在开始监听之前先写出一份完整的快照。此后每一轮，它把这一轮写下的记录封成一组，如果组的 COMMIT 要求就同步，让等待这一组的应答发出，并在需要快照时写出下一份快照的一部分。停止时，它封好并同步最后的记录，存储区则留给程序去关闭。给不保存任何东西的服务器一个存储区，或者需要存储区的服务器没有得到存储区，都是 MQTT_INVALID_ARGUMENT。

api-reference-zh.tex Page 18 November 8, 2026 13:47

表 2.4 `mqtt_net_server_options` 的字段

字段	含义
`host`	要绑定的本地地址, `NULL` 表示全部。
`port`	MQTT over TCP 的端口, 以文本表示, `NULL` 表示不用。
`tls_port`	MQTT over TLS 的端口, 或 `NULL`。
`websocket_port`	MQTT over WebSocket 的端口, 或 `NULL`。
`certificate_file`	服务器的证书链, PEM 格式, 用于 TLS。
`key_file`	服务器的私钥, PEM 格式。
`key_password`	密钥的口令, 没有口令时为 `NULL`。
`ca_file`	要求客户端证书链接到的证书; 为 `NULL` 时不要求客户端证书。
`tier`	等待分级的行为, `NULL` 表示默认值。
`send_buffer`	每个连接的系统发送缓冲区的字节数, 0 表示交给系统决定。
`store`	服务器保存会话状态的存储区, 来自 `mqtt_platform_store_open` (§4.5.2); 或者 `MQTT_STORE_NONE` (由 `base_platform.h` 定义), 表示不保存。没有什么会填写这些选项, 所以这一项也必须设置。

2.6.2 `mqtt_net_server_counters`

读取正在运行的、或最后一次运行的服务器的网络层所计的数: `tier` 中是各等待分级的计数, 即一个 `mqtt_tier_counters`; `batches` 中是发现有字节等待发送的次数; `write_calls` 中是写入套接字或 TLS 的调用次数。

```
void mqtt_net_server_counters(mqtt_net_counters *counters);
```

见 *include/base_net.h*

参数	含义
`counters` 输出	计数。
返回	无。

api-reference-zh.tex Page 19 November 8, 2026 13:47

2.7 在程序自己的循环中运行

自己搬运字节的程序从 `mqtt_server_connection_open`（§2.7.1）为每个连接得到一个索引，并把它的套接字保存在旁边。它的循环每一轮做这些事：

1. 调用 `mqtt_server_tick`（§2.7.12），它运行定时器，并说明循环可以等待多久。
2. 等待就绪的套接字，但不超过那个时长。
3. 对新连接，调用 `mqtt_server_connection_open`（§2.7.1）。没有空间时，关闭套接字。
4. 对可读的连接，只要 `mqtt_server_receive_wanted`（§2.7.3）这样说，就读入 `mqtt_server_receive_space`（§2.7.5）给出的位置，然后以读到的字节数调用 `mqtt_server_receive_commit`（§2.7.6），即使什么也没读到。
5. 对有字节要发送的连接，写出 `mqtt_server_transmit_view`（§2.7.8）的各段，然后以套接字接收的字节数调用 `mqtt_server_transmit_consume`（§2.7.9）。
6. 有存储区时，用 `mqtt_server_store_commit`（§2.10.2）封好这一轮的记录，保存它们，再用 `mqtt_server_stored`（§2.10.3）报告，好让等待它们的应答发出（第 2.10 节）。
7. 调用 `mqtt_server_next_pending`（§2.7.11），直到它给出 `MQTT_SERVER_NONE`，并对它给出的每个连接执行第 4 步和第 5 步：发给一个连接的消息来自其它连接，它的应答来自存储区，所以即使它自己的套接字没有就绪，它也可能有字节要发送，或者又有了读取的空间。
8. 当 `mqtt_server_should_close`（§2.7.10）这样说，或者套接字出错时，关闭套接字并调用 `mqtt_server_connection_closed`（§2.7.2）。

第 5.1 节的清单 `server_loop.c` 就是这样一个循环，它每一轮查看每个套接字，以保持简短。

api-reference-zh.tex Page 20 November 8, 2026 13:47

2.7.1 mqtt_server_connection_open

报告一个新的网络连接，并给它一个索引，此后有关它的每次调用都要用到这个索引。

```
mqtt_status mqtt_server_connection_open(mqtt_server *server, uint64_t now_ms,
                                        uint32_t *connection);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
now_ms 输入	时间。
connection 输出	它的索引，小于 connection_count。
返回	MQTT_OK；每个连接都在使用中时为 MQTT_QUOTA_EXCEEDED：此时关闭套接字。

这个连接有 connect_timeout_ms 的时间发送它的 CONNECT；为 0 时不限。

api-reference-zh.tex Page 21 November 8, 2026 13:47

2.7.2 mqtt_server_connection_closed

报告一个网络连接已经断开：客户端关闭了它，套接字出错，或者 mqtt_server_should_close（§2.7.10）要求关闭它。然后服务器执行协议在连接结束时要求的事，这个索引不能再使用。

```
void mqtt_server_connection_closed(mqtt_server *server, uint32_t connection,
                                   uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	它的索引。没有打开的会被忽略。
now_ms 输入	时间。
返回	无。

2.7.3 mqtt_server_receive_wanted

决定现在是否从一个连接读取。没有空间时，持有一个已知大小的报文的一部分的连接会把它移到页区，使它的块回到块池。没有东西要发送的连接则等待块，块回来时 mqtt_server_next_pending（§2.7.11）会交出它：那时再问一次。

```
bool mqtt_server_receive_wanted(mqtt_server *server, uint32_t connection);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。
返回	有空间容纳字节时为 true。

api-reference-zh.tex Page 22 November 8, 2026 13:47

不被读取的连接把它的字节留在系统的缓冲区里，缓冲区会被填满，客户端的发送就会放慢：内存不足的服务器就是这样让客户端慢下来的。

2.7.4 mqtt_server_receive_capacity

求出现在可以交给一个连接多少字节。读取不超过这个数，就为回复留出了空间，回复占用的从不多于它们所回应的报文。

```
uint32_t mqtt_server_receive_capacity(const mqtt_server *server,
                                      uint32_t connection);
```

见 *include/base_server.h*

参数	含义
`server` 输入	服务器。
`connection` 输入	连接。
返回	字节数：客户端没有读取发给它的内容时、块池的空闲块太少时，或者对没有打开的连接，为零。

api-reference-zh.tex Page 23 November 8, 2026 13:47

2.7.5 mqtt_server_receive_space

求出一个连接接下来发来的字节可以放在哪里，并为它们取得块。按顺序从套接字读入这些位置，就像一次 readv 那样，然后调用 mqtt_server_receive_commit（§2.7.6）。

```
uint32_t mqtt_server_receive_space(mqtt_server *server, uint32_t connection,
                                   mqtt_space *spaces, uint32_t max_spaces);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。
spaces 输出	这些位置，按字节填充它们的顺序排列。它们的总容量不超过 mqtt_server_receive_capacity（§2.7.4）。
max_spaces 输入	spaces 能容纳的项数。
返回	位置的数目；现在没有空间时为零。

每次调用之后都必须调用 mqtt_server_receive_commit（§2.7.6），什么也没收到时以零调用，这会交还没有写入任何东西的块。

api-reference-zh.tex Page 24 November 8, 2026 13:47

2.7.6 mqtt_server_receive_commit

接收写入 mqtt_server_receive_space（§2.7.5）所给位置的字节，把它们切分成报文，并处理每一个完整的报文：它返回时，回复以及它发给其它连接的消息都已就绪。

```
mqtt_status mqtt_server_receive_commit(mqtt_server *server, uint32_t connection,
                                       uint32_t count, uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。
count 输入	按顺序写入的字节数，可以是零。超出这些位置容量的部分不计。
now_ms 输入	时间。
返回	MQTT_OK；对没有打开的连接为 MQTT_INVALID_ARGUMENT；或者连接现在正在关闭的原因。

正在关闭的连接仍有字节要发送，例如说明原因的 CONNACK 或 DISCONNECT：继续发送，直到 mqtt_server_should_close（§2.7.10）要求关闭。

api-reference-zh.tex Page 25 November 8, 2026 13:47

2.7.7 mqtt_server_receive

从程序的缓冲区接收一个连接发来的字节，适合拿到字节时它们已经在缓冲区里的程序。它在内部调用 mqtt_server_receive_space（§2.7.5）和 mqtt_server_receive_commit（§2.7.6），所以比直接调用这两个函数多复制一次。

```
mqtt_status mqtt_server_receive(mqtt_server *server, uint32_t connection,
                                const uint8_t *data, uint32_t length,
                                uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。
data 输入	字节。
length 输入	data 中的字节数。
now_ms 输入	时间。
返回	MQTT_OK；对没有打开的连接为 MQTT_INVALID_ARGUMENT；没有空间容纳全部字节时为 MQTT_RING_FULL，此时其余的没有被接收；或者连接现在正在关闭的原因。

api-reference-zh.tex Page 26 November 8, 2026 13:47

2.7.8 mqtt_server_transmit_view

求出一个连接接下来要发送的字节，以一次 writev 可以一起发送的若干段给出：它的环的每一块一段，从页区直接发送的有效载荷一段。

```
uint32_t mqtt_server_transmit_view(mqtt_server *server, uint32_t connection,
                                   mqtt_run *runs, uint32_t max_runs);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。
runs 输出	这些段，按发送顺序排列。它们在有关这个服务器的下一次调用之前一直有效。
max_runs 输入	runs 能容纳的项数。少于实际的段数时，给出前面那些。
返回	段的数目；现在没有要发送的，或者连接没有打开时为零。

api-reference-zh.tex Page 27 November 8, 2026 13:47

2.7.9 mqtt_server_transmit_consume

报告视图中有多少字节已经发出，从它的第一段算起。

```
void mqtt_server_transmit_consume(mqtt_server *server, uint32_t connection,
                                  uint32_t count);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。没有打开的会被忽略。
count 输入	发出的字节数，可以在任何一段中结束。超出等待发送的部分不计。
返回	无。

2.7.10 mqtt_server_should_close

求出一个连接现在是否应当关闭。

```
bool mqtt_server_should_close(const mqtt_server *server, uint32_t connection);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
connection 输入	连接。
返回	它正在关闭并且没有剩下要发送的东西时为 true；没有打开时为 false。

然后关闭套接字，并调用 mqtt_server_connection_closed(§2.7.2)。

api-reference-zh.tex Page 28 November 8, 2026 13:47

2.7.11 mqtt_server_next_pending

逐个交出自上次调用以来有事要做的连接：有来自另一个连接的字节要发送，或者在等待块之后又有了读取的空间。

```
uint32_t mqtt_server_next_pending(mqtt_server *server);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
返回	一个连接；没有更多时为 MQTT_SERVER_NONE。

2.7.12 mqtt_server_tick

运行定时器：Keep Alive、对 CONNECT 的等待、遗嘱延迟间隔、会话过期间隔、消息过期，以及 MQTT 3.1 的重发。

```
uint32_t mqtt_server_tick(mqtt_server *server, uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
now_ms 输入	时间。
返回	距离它应当再次运行的毫秒数，这也是循环最多应当等待的时长。

api-reference-zh.tex Page 29 November 8, 2026 13:47

2.7.13 mqtt_server_take_activity

求出一个连接自上次调用以来发来了什么，并重新开始计。报文从它的第一个字节起计入。网络层用它区分只是维持连接的客户端和有流量的客户端，并以较低的频率等待前者。

```
mqtt_server_activity mqtt_server_take_activity(mqtt_server *server,
                                               uint32_t connection);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。
返回	MQTT_SERVER_ACTIVITY_NONE；只有 PINGREQ 报文开始到达时为 MQTT_SERVER_ACTIVITY_PINGREQ；否则为 MQTT_SERVER_ACTIVITY_OTHER。对没有打开的连接为 MQTT_SERVER_ACTIVITY_NONE。

2.7.14 mqtt_server_accepted

求出一个连接的 CONNECT 是否已被接受。

```
bool mqtt_server_accepted(const mqtt_server *server, uint32_t connection);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
connection 输入	连接。
返回	从接受它的 CONNACK 起，直到它开始关闭，都为 true。

api-reference-zh.tex Page 30 November 8, 2026 13:47

2.7.15 mqtt_server_keep_alive

求出一个连接上正在生效的 Keep Alive：客户端的，或者取代了它的 Server Keep Alive。

```
uint16_t mqtt_server_keep_alive(const mqtt_server *server,
                                uint32_t connection);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
connection 输入	连接。
返回	秒数；没有 Keep Alive，或者连接没有被接受时为 0。

2.7.16 mqtt_server_idle

求出一个连接是否什么也没持有：没有已到达的报文的任何部分，也没有要发送的东西。

```
bool mqtt_server_idle(const mqtt_server *server, uint32_t connection);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
connection 输入	连接。
返回	什么也没持有时为 true；没有打开时为 false。

api-reference-zh.tex Page 31 November 8, 2026 13:47

2.7.17 mqtt_server_disconnect

从服务器一侧关闭一个连接，在协议有办法时告诉客户端原因：MQTT 5.0 中是一个带原因码的 DISCONNECT。继续发送，直到 mqtt_server_should_close（§2.7.10）要求关闭。

```
void mqtt_server_disconnect(mqtt_server *server, uint32_t connection,
                            uint8_t reason_code);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
connection 输入	连接。没有打开的会被忽略。
reason_code 输入	原因码，来自 v5_rc.h，例如 MQTT5_RC_ADMINISTRATIVE_ACTION。
返回	无。

api-reference-zh.tex Page 32 November 8, 2026 13:47

2.8 来自程序的消息

2.8.1 mqtt_server_publish

从服务器一侧发布一条应用消息，发给订阅了它的主题的每个会话，就像某个客户端发布了它一样。字节会被复制到保存它们的地方，所以它返回后就可以重用。

```
mqtt_status mqtt_server_publish(mqtt_server *server,
                                const mqtt_utf8 *topic_name,
                                const mqtt_payload *payload, uint8_t qos,
                                bool retain, uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
topic_name 输入	主题名，必须有效，并且不含通配符。
payload 输入	有效载荷。
qos 输入	QoS：0、1 或 2。PUBLISH 报文不可以是 3。
retain 输入	true 表示让它成为它的主题的保留消息。有效载荷为零字节时，它改为删除那条消息，但仍会发给订阅者。
now_ms 输入	时间。
返回	MQTT_OK；MQTT_TOPIC_NAME_INVALID；主题名不是有效的 UTF-8 时为 MQTT_MALFORMED_PACKET；服务器没有初始化、qos 大于 2，或者要求保留而 retain_available 为 false 时为 MQTT_INVALID_ARGUMENT；或者 MQTT_QUOTA_EXCEEDED，见下文。

MQTT_QUOTA_EXCEEDED 有三种原因，只有第一种表示什么也没做。主题名放不进连接的环时，什么也没有发出。没有空间保留这条消息，或者没有空间为某个会话排队时，有空间的那些会话已经收到了它，所以再发布一次会让它们收到第二份。没有找到空间的保留消息仍会取代旧的那条，旧的就没有了。

api-reference-zh.tex Page 33 November 8, 2026 13:47

2.8.2 mqtt_server_retained_next

逐条交出保留消息，供自己把它们保存下来、以便重启后仍然存在的程序使用。有存储区的服务器把它们和会话一起保存在那里（第 2.9 节）。游标从零开始。

```
bool mqtt_server_retained_next(const mqtt_server *server, uint64_t now_ms,
                               uint32_t *cursor, mqtt_server_retained *message);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
now_ms 输入	时间，用于计算每条消息的过期还剩多少。
cursor 输入输出	从哪里开始找：零表示从头开始。
message 输出	消息（表 2.5）。
返回	有一条时为 true，没有更多时为 false。

它交出的内容和游标只在服务器下一次运行之前有效：在调用其它任何函数之前保存它们。

表 2.5 mqtt_server_retained 的字段

字段	含义
topic_name	主题名。
properties	它的属性，已编码，不含它们的长度。
payload	有效载荷。
expiry_left_ms	它的消息过期间隔还剩下的部分；没有时为 0。
qos	发布它时所用的 QoS。

api-reference-zh.tex Page 34 November 8, 2026 13:47

2.8.3 mqtt_server_retained_restore

放回一条由 mqtt_server_retained_next（§2.8.2）交出的保留消息，时机是在 mqtt_server_init(§2.5.3)之后、任何客户端连接之前。

```
mqtt_status mqtt_server_retained_restore(mqtt_server *server,
                                         const mqtt_server_retained *message,
                                         uint64_t now_ms);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
message 输入	消息，与保存时一样。
now_ms 输入	时间，它的过期从这时重新开始计算。
返回	MQTT_OK；MQTT_TOPIC_NAME_INVALID；主题名不是有效的 UTF-8 时为 MQTT_MALFORMED_PACKET；服务器没有初始化、retain_available 为 false、有效载荷为空、QoS 大于 2，或者属性不是 PUBLISH 可以携带的，或其中有主题别名、订阅标识符或消息过期间隔时，为 MQTT_INVALID_ARGUMENT；或者 MQTT_QUOTA_EXCEEDED。

2.9 让会话状态在重启后仍在

服务器可以把停止时本会丢失的东西保存在一个存储区中：它的会话（连同其订阅和遗嘱消息）、等待发给会话的消息和在途消息、保留消息，以及它收到的 QoS 2 消息的报文标识符。在同一个存储区上再次启动时，它从那里继续，继续自己会话的客户端会发现会话和原来一样。第 5.1 节的 server_store.c 就是这样的服务器。

- **日志与快照**。每个改动是一条记录，在发生时交给 store_append。快照以记录的形式保存整个状态，让日志可以重新开始。它保存为两份或更多副本，依次写出，所以无论另一份出了什么事，总有一份是完整的(第 4.5 节)。
- **保存之前不确认**。一轮的记录被封成一组。告诉客户端某样东西已被保存的应答，

api-reference-zh.tex Page 35 November 8, 2026 13:47

要等程序说明它所在的组已经存储才发出：CONNACK、PUBACK、PUBREC、PUBREL、PUBCOMP、SUBACK、UNSUBACK，以及第一次发出的 QoS 2 PUBLISH。它在连接的环中等待，所以排在它前面的字节照样发出。

- **每类记录经受得住什么。** `store_sessions` 和 `store_messages` 各取表 4.4 中的一个值：断电、进程崩溃，或者什么都不经受。其它任何值都会让 `mqtt_server_init`（§2.5.3）拒绝这个配置。
- **要么全部，要么全无。** 恢复时，每条记录都像客户端发来的内容一样受到检查。损坏的、由更新版本写入的，或者超出这个配置和这些内存容量的存储区，一点也不会被采用：服务器从空状态启动，存储区被移到一旁，从不删除。会话从不只恢复一半。

使用 `platform/` 时，程序用 `mqtt_platform_store_open`（§4.5.2）打开存储区，在 `mqtt_server_init`（§2.5.3）之前用 `mqtt_net_server_store_config`（§2.9.1）设置配置，用 `mqtt_net_server_restore`（§2.9.2）放回存储区保存的东西，再在选项的 `store` 中把存储区交给 `mqtt_net_server_run`（§2.6.1）。其余的由循环完成，循环返回后，程序用 `mqtt_platform_store_close`（§4.5.3）关闭存储区。有自己循环的程序改用第 2.10 节的调用。

2.9.1 mqtt_net_server_store_config

设置服务器的配置，让它把自己的会话状态保存在 `platform/` 的一个存储区中。记录都交给 `mqtt_platform_store_append`，它得到的标签就是存储区，所以 `user_tag` 变成存储区，每个回调得到的也是它。会话和消息都按经受得住断电的要求保存，程序可以在这次调用之后把它降为 `MQTT_STORE_WRITTEN`。快照最晚在日志写满存储区一个位置的一半时进行，所以日志段永远不会写满它的位置。

```
void mqtt_net_server_store_config(mqtt_server_config *config, int32_t store);
```

见 *include/base_net.h*

参数	含义
`config` 输入输出	配置，在 `mqtt_server_config_default`（§2.5.1）之后、`mqtt_server_init`（§2.5.3）之前。
`store` 输入	存储区，来自 `mqtt_platform_store_open`（§4.5.2）。
返回	无。

api-reference-zh.tex Page 36 November 8, 2026 13:47

2.9.2 mqtt_net_server_restore

把存储区保存的东西放回刚初始化的服务器，在 mqtt_net_server_run（§2.6.1）之前调用。损坏的、来自更新版本的，或者超出这个服务器容量的存储区，会原封不动地移到一旁，控制台上有一行说明原因和去处，服务器从空状态启动。

```
mqtt_status mqtt_net_server_restore(mqtt_server *server, int32_t store,
                                    mqtt_server_restore_entry *map,
                                    uint32_t map_count,
                                    mqtt_store_fault *fault);
```

见 *include/base_net.h*

参数	含义
server 输入输出	服务器，用 mqtt_net_server_store_config（§2.9.1）的配置初始化，并且还什么都没有。
store 输入	存储区。
map 输出	一张表的空间，用来按存储区中的编号找到消息和投递：存储的 message_count 与 delivery_count 之和那么多项总是够用。
map_count 输入	map 中的项数。
fault 输出	被移到一旁的存储区出了什么问题（表 4.7），或者 MQTT_STORE_FAULT_NONE。
返回	服务器可以运行（已恢复或为空）时为 MQTT_OK；按 mqtt_server_restore（§2.10.1）的结果给出 MQTT_INVALID_ARGUMENT 或 MQTT_INVALID_STATE；损坏的存储区无法移到一旁时为 MQTT_INVALID_STATE，此时服务器不能在它上面运行。

存储区被移到一旁之后，返回的也是 MQTT_OK，所以只有 fault 能区分这两种情况。读取经过 platform/ 自己的一个静态缓冲区，大小为 MQTT_SERVER_RESTORE_SPACE 字节，约 128 KiB。

2.10 在程序自己的循环中使用存储区

自己保存存储区的程序（介质不限）在配置中设置 store_append 和两个持久程度，user_tag 则供自己使用。记录由它保存，按第 4.5 节的部分和位置存放：快照的各份副本，以及

api-reference-zh.tex Page 37 November 8, 2026 13:47

日志的两个位置，每一代轮流使用一个。然后：

1. 在 `mqtt_server_init`（§2.5.3）之后，用 `mqtt_server_restore`（§2.10.1）放回存储区保存的东西。如果它给出 `MQTT_STORE_DAMAGED`，就把存储区移到一旁，让它重新从空开始。
2. 在第一个连接之前，做一次完整的快照：调用 `mqtt_server_snapshot_begin`（§2.10.7），用快照的头部开始第一份副本，加入 `mqtt_server_snapshot_next`（§2.10.8）给出的每条记录，直到 `MQTT_NOT_FOUND`，然后同步它。如果 `new_segment` 为 true，就清空新一代的日志位置，在那里写入日志的头部，并同步。写出其它副本，然后调用 `mqtt_server_snapshot_done`（§2.10.9）。
3. 每一轮在处理完事件之后，用 `mqtt_server_store_commit`（§2.10.2）封好这一轮的记录，如果 COMMIT 要求就同步，然后用 `mqtt_server_stored`（§2.10.3）报告它们；同步失败时则调用 `mqtt_server_store_failed`（§2.10.4）。应答现在可以发出的连接，会从 `mqtt_server_next_pending`（§2.7.11）出来。
4. 当 `mqtt_server_snapshot_due`（§2.10.6）这样说时，开始一次快照，并立即开始它的日志段，在服务器再次运行之前。然后每一轮写出它的一部分记录，在 `between` 为 true 的地方停下。服务器在这之间照常运行，其间的改动写入新的日志段。
5. 当 `mqtt_server_store_open`（§2.10.5）说有记录还没有封组时，不要等待套接字：先把它们封好。

api-reference-zh.tex Page 38 November 8, 2026 13:47

2.10.1 mqtt_server_restore

把存储区保存的东西放回刚初始化的服务器。先读取并检查快照的每份副本和日志的两个位置，然后应用其日志段能够接续的最新一份副本，再应用这些日志段，直到各自最后一个 COMMIT。每条记录都像客户端发来的内容一样受到检查，并且必须符合这个配置。

```
mqtt_status mqtt_server_restore(mqtt_server *server,
                                const mqtt_store_source *source,
                                mqtt_server_restore_entry *map,
                                uint32_t map_count, uint8_t *buffer,
                                uint32_t size, uint64_t now_ms,
                                mqtt_store_fault *fault);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器，带存储区初始化，并且还什么都没有。
source 输入	存储区，有 1 到 MQTT_SERVER_RESTORE_COPIES(8)份副本(表 4.6)。
map 输出	存储区编号表的空间：存储的 message_count 与 delivery_count 之和那么多项总是够用。
map_count 输入	map 中的项数。
buffer 输出	读取用的空间，至少 MQTT_SERVER_RESTORE_MINIMUM (8,192)个字节。除消息之外的每条记录都必须放得进去，而在 MQTT_SERVER_RESTORE_SPACE 中任何记录都放得进去。
size 输入	buffer 中的字节数。
now_ms 输入	时间。
fault 输出	结果为 MQTT_STORE_DAMAGED 时，说明出了什么问题。
返回	MQTT_OK；MQTT_STORE_DAMAGED；服务器没有初始化、不保存任何东西或某个参数不对时为 MQTT_INVALID_ARGUMENT；服务器已经有东西时为 MQTT_INVALID_STATE。

它要么全部完成，要么什么都不做。只要有任何问题，服务器就会重新初始化为空，由

api-reference-zh.tex Page 39 November 8, 2026 13:47

程序把存储区移到一旁：这样，来自更新版本的存储区，或者不符合这个配置或这些内存的存储区，以后可以由能够接受它的服务器读取。什么都没有的存储区给出 MQTT_OK，并且不改变任何东西。各种时间恢复为当时剩下的部分，再减去来源的 downtime_ms。会话恢复时不带连接：存储区最后一次写入时有连接的会话，算作那时失去了连接。在 mqtt_server_snapshot_begin（§2.10.7）之前什么都不写，而 mqtt_server_snapshot_due（§2.10.6）随后就会要求它。

2.10.2 mqtt_server_store_commit

把上次调用以来写下的记录封成一组，加上一条 COMMIT 记录。等待它们的应答，在程序用 mqtt_server_stored（§2.10.3）报告这一组已经存储后发出。COMMIT 用它的持久程度说明这一组是否先要同步。循环在每一轮处理完事件之后调用一次，好让一次同步覆盖这一轮到来的所有东西。

```
mqtt_status mqtt_server_store_commit(mqtt_server *server, uint64_t now_ms,
                                     uint64_t *sequence);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
now_ms 输入	时间。
sequence 输出	封好的最后一组，也就是要报告的那一组。
返回	MQTT_OK，没有什么可封时也是；服务器没有初始化或不保存任何东西时为 MQTT_INVALID_ARGUMENT；日志已断时为 MQTT_INVALID_STATE，此时 sequence 是断开之前封好的最后一组。

api-reference-zh.tex Page 40 November 8, 2026 13:47

2.10.3 mqtt_server_stored

报告直到某一组为止的每一组都已按其 COMMIT 的要求存储，好让等待它们的应答发出。每个又可以发送的连接，都由 mqtt_server_next_pending（§2.7.11）交出。

```
void mqtt_server_stored(mqtt_server *server, uint64_t sequence);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
sequence 输入	已存储的最后一组。超过已封好的最后一组时，按那一组计算。
返回	无。

2.10.4 mqtt_server_store_failed

报告记录可能已经丢失，例如同步失败时。日志于是断开：不再向它写入任何东西，每个应答都要等待，直到新一代的快照完整为止。

```
void mqtt_server_store_failed(mqtt_server *server);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
返回	无。

api-reference-zh.tex Page 41 November 8, 2026 13:47

2.10.5 mqtt_server_store_open

查明是否写下了还没有 COMMIT 封好的记录。发现有的循环，在等待之前调用 mqtt_server_store_commit(§2.10.2)。

```
bool mqtt_server_store_open(const mqtt_server *server);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
返回	有时为 true。

2.10.6 mqtt_server_snapshot_due

查明是否应该开始一次快照：这一代还没有完整的快照，日志已断，或者日志段的长度已经超过了 store_snapshot_bytes 字节。

```
bool mqtt_server_snapshot_due(const mqtt_server *server);
```

见 *include/base_server.h*

参数	含义
server 输入	服务器。
返回	应该开始、并且没有快照在进行中时为 true。

api-reference-zh.tex Page 42 November 8, 2026 13:47

2.10.7 mqtt_server_snapshot_begin

开始一次快照，随之开始新一代的日志段。这次调用之后的每条记录都属于新的日志段，所以程序要先用日志的头部开始这个日志段，再让服务器继续运行。之前的每一组都必须已经用 mqtt_server_store_commit(§2.10.2)封好。

```
mqtt_status mqtt_server_snapshot_begin(mqtt_server *server, uint64_t now_ms,
                                       uint8_t *snapshot_header,
                                       uint8_t *journal_header,
                                       bool *new_segment);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
now_ms 输入	时间。
snapshot_header 输出	快照头部的 MQTT_STORE_HEADER_SIZE(58)个字节的空间。
journal_header 输出	日志段头部的 MQTT_STORE_HEADER_SIZE 个字节的空间。
new_segment 输出	开始一个新的日志段时为 true。
返回	MQTT_OK；服务器没有初始化或不保存任何东西时为 MQTT_INVALID_ARGUMENT；有组还没封好或已有快照在进行中时为 MQTT_INVALID_STATE。

被放弃的快照会在同一代中重做，沿用同一个日志段：此时 new_segment 为 false，日志的头部不写。日志中出现断档之后，新的日志段会标明它接在断档之后，在快照完整之前不发出任何应答。

api-reference-zh.tex Page 43 November 8, 2026 13:47

2.10.8 mqtt_server_snapshot_next

给出进行中的快照的下一条记录。两次调用之间服务器可以运行，快照仍然正确，因为每条记录说明某样东西是什么，而其间的改动在日志里。同一个会话的记录一条接一条地给出；如果在它们全部给出之前日志被写入，那个会话就从头再来：每一轮有预算的循环，因此在 between 为 true 的地方停下。

```
mqtt_status mqtt_server_snapshot_next(mqtt_server *server, uint8_t *space,
                                      uint32_t size, mqtt_store_record *record,
                                      bool *between);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
space 输出	记录的固定字段和主题过滤器放在这里，在下一次调用之前有效。
size 输入	space 中的字节数，至少 MQTT_SERVER_SNAPSHOT_SPACE（128）。再多 65,535 个，任何记录都放得下。
record 输出	记录（表 4.5）。它的各段在下一次调用之前有效。
between 输出	下一次调用开始一条新消息或一个新会话时为 true。
返回	有记录时为 MQTT_OK；最后一条已经给出后为 MQTT_NOT_FOUND；记录放不进 space 时为 MQTT_BUFFER_TOO_SMALL，此时快照必须放弃；没有快照在进行中时为 MQTT_INVALID_STATE。

api-reference-zh.tex Page 44 November 8, 2026 13:47

2.10.9 mqtt_server_snapshot_done

报告快照在每份副本中都已完整，于是下一次快照开始新的一代。日志中出现断档之后，等待这次快照的应答，在它们的组存储之后发出。

```
void mqtt_server_snapshot_done(mqtt_server *server);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
返回	无。

2.10.10 mqtt_server_snapshot_abort

放弃进行中的快照。下一次快照属于同一代，沿用同一个日志段。

```
void mqtt_server_snapshot_abort(mqtt_server *server);
```

见 *include/base_server.h*

参数	含义
server 输入输出	服务器。
返回	无。

2.11 检查运行中的服务器

服务器在运行中计数， 记在它的 statistics 字段中的 mqtt_server_statistics 里，表 2.6 列出了各项。这些计数只供读取，永远不要写入。

api-reference-zh.tex Page 45 November 8, 2026 13:47

表 2.6 `mqtt_server_statistics` 的字段

字段	所计的数
`connections_accepted`	以 CONNACK 接受的连接。
`messages_received`	从客户端收到的应用消息。
`messages_sent`	发给客户端的 PUBLISH 报文。
`messages_dropped`	因队列或池已满而丢弃的消息。
`protocol_errors`	因报文错误而关闭的连接。
`messages_refused`	没能接收的 QoS 0 消息，因为无法告知发送它的客户端。
`replies_lost`	因空间不足而没能发出的、说明连接为何关闭的报文。
`receive_throttled`	因空间不足而停止读取某个连接的次数。
`records_written`	交给 `store_append` 的记录。
`records_refused`	被 `store_append` 拒绝的记录，每一条都使日志中断。

2.11.1 `mqtt_server_pool_audit`

检查块池的每一块要么空闲，要么恰好在一个使用中的连接的一个环里，并且各项计数一致。它要遍历每一块，所以用于测试和排查故障，而不是每一轮都用。

```
bool mqtt_server_pool_audit(const mqtt_server *server, uint8_t *marks);
```

见 *include/base_server.h*

参数	含义
`server` 输入	服务器。
`marks` 输入输出	块池的每一块一个字节，全部为零。
返回	一致时为 true。

3

客户端

客户端一次连接一个服务器，使用 MQTT 3.1、3.1.1 或 5.0。只要使用的是同一个 `mqtt_client`，它就把会话状态从一个网络连接保持到下一个：没有得到确认的 QoS 1 和 2 消息，会在 `mqtt_client_connect`(§3.6.1)继续会话时再次发送。有存储区时，它还能让会话状态在重启后仍在(第 3.10 节)。它通过一个回调把服务器发来的东西告诉程序。

3.1 概览

表 3.1 客户端的函数

函数	作用	§
初始化		
`mqtt_client_init`	初始化客户端	3.5.1
`mqtt_client_assume_session`	说明它有一个来自上一次运行的会话	3.5.2
连接		
`mqtt_client_connect`	发送 CONNECT	3.6.1
`mqtt_client_auth`	发送 AUTH	3.6.2
`mqtt_client_disconnect`	发送 DISCONNECT，并开始关闭	3.6.3
`mqtt_client_should_close`	说明何时关闭套接字	3.6.4
`mqtt_client_connection_lost`	报告连接已经断开	3.6.5
消息和订阅		
`mqtt_client_publish`	发送一条应用消息	3.7.1

续页

api-reference-zh.tex Page 48 November 8, 2026 13:47

表 3.1 客户端的函数(续)

函数	作用	§
`mqtt_client_subscribe`	发送 SUBSCRIBE	3.7.2
`mqtt_client_unsubscribe`	发送 UNSUBSCRIBE	3.7.3
用 `platform/` 运行		
`mqtt_net_client_open`	打开网络连接	3.8.1
`mqtt_net_client_step`	搬运字节, 稍作等待	3.8.2
`mqtt_net_client_close`	关闭网络连接	3.8.3
在程序自己的循环中运行		
`mqtt_client_receive_capacity`	说明可以交给它多少字节	3.9.1
`mqtt_client_receive_space`	给出读入字节的位置	3.9.2
`mqtt_client_receive_commit`	接收读入其中的字节	3.9.3
`mqtt_client_receive`	从程序的缓冲区接收字节	3.9.4
`mqtt_client_transmit_view`	给出要发送的字节	3.9.5
`mqtt_client_transmit_consume`	说明其中有多少已经发出	3.9.6
`mqtt_client_tick`	运行定时器	3.9.7
保存会话状态		
`mqtt_client_store_init`	给客户端一个存储区	3.10.1
`mqtt_net_client_store_config`	为 `platform/` 的存储区填写它的配置	3.10.2
`mqtt_net_client_restore`	放回那个存储区保存的东西	3.10.3
`mqtt_client_store_identifier`	给出它保存的客户端标识符	3.10.4
`mqtt_client_restore`	放回存储区保存的东西, 介质不限	3.11.1
`mqtt_client_store_commit`	把一轮的记录封成一组	3.11.2
`mqtt_client_stored`	说明各组已经存储, 它们的应答可以发出	3.11.3
`mqtt_client_store_failed`	说明记录可能已经丢失	3.11.4
`mqtt_client_store_open`	说明是否有记录等待封组	3.11.5
`mqtt_client_snapshot_due`	说明是否应该开始一次快照	3.11.6
`mqtt_client_snapshot_begin`	开始一次快照	3.11.7

续页

api-reference-zh.tex Page 49 November 8, 2026 13:47

表 3.1 客户端的函数(续)

函数	作用	§
mqtt_client_snapshot_next	给出它的下一条记录	3.11.8
mqtt_client_snapshot_done	说明它的第一份副本已经完整	3.11.9
mqtt_client_snapshot_abort	放弃它	3.11.10

3.2 客户端的生命周期

1. 在一个 mqtt_client_storage 中提供内存和事件回调（第 3.3 节），并用 mqtt_client_init(§3.5.1)初始化客户端。
2. 要让会话状态在重启后仍在，就给它一个存储区，并放回存储区保存的东西（第 3.10 节）。
3. 打开一个网络连接:用 mqtt_net_client_open(§3.8.1),或者用程序自己的连接。
4. 用 mqtt_client_connect（§3.6.1）发送 CONNECT，然后搬运字节，直到 MQTT_CLIENT_EVENT_CONNACK 事件说明它是否被接受。
5. 发布、订阅和取消订阅。每个回应都以事件的形式到来。
6. 以 mqtt_client_disconnect(§3.6.3)结束,搬运字节直到该关闭的时候,然后关闭:用 mqtt_net_client_close（§3.8.3），或者在程序自己的循环中用 mqtt_client_connection_lost(§3.6.5)。
7. 以后要继续,就打开一个新连接,用同一个客户端再次发送 CONNECT。

3.3 内存

mqtt_client_storage 指明客户端将用到的每个数组，以及接收客户端事件的那个回调。第 5.2 节的清单 client_setup.c 填写了一个。

api-reference-zh.tex Page 50 November 8, 2026 13:47

表 3.2 `mqtt_client_storage` 的字段

字段	应提供
`on_event`、`user_tag`	事件回调，`NULL` 表示忽略事件，以及交给它的标签（第 3.4 节）。
`receive_bytes`、`receive_size`	至少 5 个字节，用于分几部分到达的报文。它的大小就是客户端能接收的最大报文，并作为 Maximum Packet Size 发给服务器。
`pool_bytes`、`pool_next`	`pool_block_count` 乘以 `pool_block_size` 个字节，总共至少 256 个，以及每块一个 `uint32_t`：等待发送的字节。
`pool_block_size`、`pool_block_count`	一块的字节数，至多 4,096，以及块的数目。
`messages`、`message_count`	每条暂存到得到确认为止的消息一个 `mqtt_client_message`。
`incoming`、`incoming_count`	来自服务器、同时进行中的每条 QoS 2 消息一个 `uint16_t`：即它的 Receive Maximum。
`pending`、`pending_count`	每个等待回应的 SUBSCRIBE 或 UNSUBSCRIBE 一个 `mqtt_client_pending`。
`aliases`、`alias_count`	服务器可以设置的每个主题别名一个 `mqtt_client_alias`：即它的 Topic Alias Maximum。没有时为 `NULL` 和 0。
`arena_bytes`、`arena_states`、`arena_units`	`arena_units` 乘以 `MQTT_ARENA_UNIT`（64）个字节，以及每个单元一个字节，用于暂存的报文和主题别名的主题名。

3.4 事件

服务器发来的东西以事件的形式，通过存储中的回调到达程序。回调在接收这些字节的那次调用中被调用：`mqtt_net_client_step`（§3.8.2），或者在程序自己的循环中是 `mqtt_client_receive_commit`（§3.9.3）。

api-reference-zh.tex Page 51 November 8, 2026 13:47


```
typedef void (*mqtt_client_event_function)(uint32_t user_tag,
                                           const mqtt_client_event *event);
```

见 *include/base_client.h*

事件的 type 说明它的哪个字段被设置了（表 3.3）。这些指针，以及它们指向的每个字节，都只在回调期间有效：以后要用的东西请复制下来。回调可以调用 mqtt_client_publish（§3.7.1）、mqtt_client_subscribe（§3.7.2）以及其它把报文排队的函数。

表 3.3 客户端的事件

类型	设置的字段	何时
MQTT_CLIENT_EVENT_CONNACK	connack	CONNACK 到达。连接被接受时它的 reason_code 为 0；它的 session_present 和属性也在其中。
MQTT_CLIENT_EVENT_AUTH	reason	在增强认证中，一个 AUTH 到达。
MQTT_CLIENT_EVENT_MESSAGE	publish	一条应用消息到达：它的 topic_name、payload、qos、retain 和属性。确认由库发送。
MQTT_CLIENT_EVENT_PUBLISHED	response	本客户端的一条 QoS 1 或 2 消息结束了它的流程：PUBACK 或 PUBCOMP，带有它的 packet_identifier 和 reason_code，或者一次拒绝。
MQTT_CLIENT_EVENT_SUBACK	subscribed	一个 SUBACK 到达：在 reason_codes 中，每个主题过滤器一个原因码。
MQTT_CLIENT_EVENT_UNSUBACK	subscribed	一个 UNSUBACK 到达。
MQTT_CLIENT_EVENT_PINGRESP	–	一个 PINGRESP 到达。
MQTT_CLIENT_EVENT_DISCONNECT	reason	一个 DISCONNECT 到达：服务器发来的，带有它的原因码。

api-reference-zh.tex Page 52 November 8, 2026 13:47

在报文中带有原因码的每个版本里，大于或等于 MQTT5_RC_FAILURE_MIN（0x80）的原因码都表示失败。

3.5 初始化

3.5.1 mqtt_client_init

初始化一个客户端。对内存的描述会被复制：数组本身从此归客户端所有。

```
mqtt_status mqtt_client_init(mqtt_client *client,
                             const mqtt_client_storage *storage);
```

见 *include/base_client.h*

参数	含义
client 输出	客户端。
storage 输入	内存，以及回调。
返回	MQTT_OK；存储违反表 3.2 的某条规则时为 MQTT_INVALID_ARGUMENT。

api-reference-zh.tex Page 53 November 8, 2026 13:47

3.5.2 mqtt_client_assume_session

说明这个客户端有一个来自上一次运行、其中没有在途消息的会话。不调用它的话，收到 Session Present 为 1 的 CONNACK 时连接会被关闭，因为协议要求没有会话状态的客户端这样做。没有存储区、重新启动、并想保留之前所做订阅的程序，在 `mqtt_client_connect` （§3.6.1） 之前调用它。有存储区的客户端完全不需要它： `mqtt_net_client_restore`(§3.10.3)会放回它的会话，连同在途消息。

```
void mqtt_client_assume_session(mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
`client` 输入输出	客户端。
返回	无。

api-reference-zh.tex Page 54 November 8, 2026 13:47

3.6 连接

3.6.1 mqtt_client_connect

在一个新的网络连接上把 CONNECT 排队。除非报文自己设置了，它会加上存储所能支持的 Receive Maximum、Maximum Packet Size 和 Topic Alias Maximum。当 MQTT_CLIENT_EVENT_CONNACK 事件这样说时，连接才算被接受。

```
mqtt_status mqtt_client_connect(mqtt_client *client, const mqtt_connect *packet,
                                uint64_t now_ms);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端，没有进行中的连接。
packet 输入	CONNECT（表 3.4）。它引用的字节在调用之后不再使用。
now_ms 输入	时间。
返回	MQTT_OK；已经有连接在进行中时为 MQTT_INVALID_STATE；没有空间容纳它时为 MQTT_BUFFER_TOO_SMALL。

从 mqtt_connect_clear（§4.3.2）开始，它会略去每个可选部分，然后设置连接所需要的。

表 3.4 mqtt_connect 的字段

字段	含义
version	MQTT_VERSION_31、 MQTT_VERSION_311 或 MQTT_VERSION_5。
clean_start	MQTT 5.0 中的 Clean Start，之前版本中的 Clean Session。
keep_alive	Keep Alive，以秒计，0 表示不用。
client_identifier	客户端标识符。在 3.1.1 和 5.0 中，为空表示请服务器分配一个。

续页

api-reference-zh.tex Page 55 November 8, 2026 13:47

表 3.4 mqtt_connect 的字段(续)

字段	含义
user_name_flag、user_name	一个用户名,如果设置了该标志。
password_flag、password	一个密码,以二进制数据表示,如果设置了该标志。
will_flag、will_topic、will_payload、will_qos、will_retain	一条遗嘱消息,如果设置了该标志。
properties	MQTT 5.0 的 CONNECT 属性(第 4.2 节)。
will_properties	MQTT 5.0 的遗嘱属性。

3.6.2 mqtt_client_auth

在 CONNECT 带有认证方法时,为 MQTT 5.0 的增强认证把一个 AUTH 排队:原因码为 0x18(继续认证),或者在连接被接受之后为 0x19(重新认证)。它的属性中的认证方法必须与 CONNECT 的相同。

```
mqtt_status mqtt_client_auth(mqtt_client *client, const mqtt_reason *packet);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
packet 输入	AUTH:它的原因码和属性。
返回	MQTT_OK;或者它无法排队的原因,例如 MQTT_INVALID_STATE。

api-reference-zh.tex Page 56 November 8, 2026 13:47

3.6.3 mqtt_client_disconnect

把一个 DISCONNECT 排队，并开始关闭。继续搬运字节，直到 mqtt_client_should_close(§3.6.4)要求关闭。

```
mqtt_status mqtt_client_disconnect(mqtt_client *client,
                                   const mqtt_reason *packet);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
packet 输入	DISCONNECT, NULL 表示普通的那种。在 MQTT 5.0 中, 原因码 0x04 请服务器发布遗嘱消息。
返回	MQTT_OK; 或者它无法排队的原因, 这种情况下它照样关闭。

3.6.4 mqtt_client_should_close

求出连接现在是否应当关闭。

```
bool mqtt_client_should_close(const mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入	客户端。
返回	它正在关闭并且没有剩下要发送的东西时为 true。

api-reference-zh.tex Page 57 November 8, 2026 13:47

3.6.5 mqtt_client_connection_lost

报告网络连接已经断开，无论是不是程序关闭的。会话状态保留下来，供下一次 mqtt_client_connect(§3.6.1)使用。mqtt_net_client_close(§3.8.3)会调用它。

```
void mqtt_client_connection_lost(mqtt_client *client, uint64_t now_ms);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
now_ms 输入	时间。
返回	无。

api-reference-zh.tex Page 58 November 8, 2026 13:47

3.7 消息和订阅

3.7.1 mqtt_client_publish

发送一条应用消息。它的 QoS、主题名和属性在这里检查，并且它会被复制，所以它返回后字节就可以重用。

```
mqtt_status mqtt_client_publish(mqtt_client *client, mqtt_publish *packet,
                                uint64_t now_ms);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
packet 输入输出	PUBLISH（表 3.5）。对大于 0 的 QoS，它的报文标识符在这里设置。
now_ms 输入	时间。
返回	MQTT_OK；或者它没有被接受的原因，这种情况下什么也没有保留。MQTT_QUOTA_EXCEEDED 表示暂时没有空间：等一些消息结束它们的流程后再试。

QoS 0 消息只在已被接受的连接上发送。QoS 1 或 2 的消息会暂存到它的流程结束，并且可以在离线时交给客户端，只要 mqtt_client_connect（§3.6.1）已经至少调用过一次、版本已经确定：它随下一个连接发出，除非那个服务器的限制不允许，这时 MQTT_CLIENT_EVENT_PUBLISHED 会以那个限制的原因码报告它。QoS 1 或 2 的消息必须有主题名，即使带有主题别名也一样，因为它可能在另一个连接上再次发送。

api-reference-zh.tex Page 59 November 8, 2026 13:47

表 3.5 `mqtt_publish` 的字段

字段	含义
`topic_name`	主题名：不含通配符。
`payload`	有效载荷，可以是任何字节。
`qos`	0、1 或 2，不超过服务器的 Maximum QoS。
`retain`	true 表示让它成为它的主题的保留消息。
`dup`	保持为 false：库在重发时设置它。
`packet_identifier`	由库设置。
`properties`	MQTT 5.0 的 PUBLISH 属性，例如 Content Type 或消息过期间隔（第 4.2 节）。

所有字节都为零的 `mqtt_publish` 没有属性，所以程序可以用 `memset` 清零一个，再设置它需要的，就像 `client_publish.c` 那样。

api-reference-zh.tex Page 60 November 8, 2026 13:47

3.7.2 mqtt_client_subscribe

在连接被接受之后，把一个 SUBSCRIBE 排队。SUBACK 以 MQTT_CLIENT_EVENT_SUBACK 事件到来，带有同一个报文标识符。

```
mqtt_status mqtt_client_subscribe(mqtt_client *client,
                                  const mqtt_client_subscription *entries,
                                  uint32_t count,
                                  const mqtt5_properties *properties,
                                  uint16_t *packet_identifier);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
entries 输入	各个主题过滤器，每个带有它的选项（表 3.6）。
count 输入	项数，至少为 1。
properties 输入	MQTT 5.0 的 SUBSCRIBE 属性，例如订阅标识符；NULL 表示没有。
packet_identifier 输出	回应将带有的报文标识符。
返回	MQTT_OK；连接被接受之前为 MQTT_INVALID_STATE；或者它没有被接受的原因，例如 MQTT_TOPIC_FILTER_INVALID。

mqtt_client_subscription 包含主题过滤器 topic_filter 和它的 options：options 是 MQTT 5.0 的订阅选项字节，在之前的版本中只是请求的 QoS：

api-reference-zh.tex Page 61 November 8, 2026 13:47

表 3.6 options 的各位, 来自 v5_subscribe.h

位	宏	含义
0–1	MQTT5_MAXIMUM_QOS_MASK	发送时所用的最高 QoS。
2	MQTT5_NO_LOCAL	不要本客户端自己的消息。
3	MQTT5_RETAIN_AS_PUBLISHED	保持发布时的 RETAIN 标志。
4–5	MQTT5_RETAIN_HANDLING_MASK	是否发送保留消息: 0 每次订阅都发送, 1 只在新订阅时发送, 2 从不发送。

3.7.3 mqtt_client_unsubscribe

在连接被接受之后,把一个 UNSUBSCRIBE 排队。UNSUBACK 以 MQTT_CLIENT_EVENT_UNSUBACK 事件到来。

```
mqtt_status mqtt_client_unsubscribe(mqtt_client *client,
                                    const mqtt_utf8 *filters, uint32_t count,
                                    const mqtt5_properties *properties,
                                    uint16_t *packet_identifier);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
filters 输入	各个主题过滤器。
count 输入	项数, 至少为 1。
properties 输入	MQTT 5.0 的 UNSUBSCRIBE 属性, NULL 表示没有。
packet_identifier 输出	回应将带有的报文标识符。
返回	MQTT_OK; 连接被接受之前为 MQTT_INVALID_STATE; 或者它没有被接受的原因。

api-reference-zh.tex Page 62 November 8, 2026 13:47

3.8 用 platform/ 运行

3.8.1 mqtt_net_client_open

打开客户端的网络连接，经由 TCP，需要时加上 TLS。在调用 mqtt_client_connect (§3.6.1)之前，客户端一直处于离线状态。

```
mqtt_status mqtt_net_client_open(mqtt_net_client *network, mqtt_client *client,
                                 const char *host, const char *port,
                                 const mqtt_net_client_tls *tls);
```

见 *include/base_net.h*

参数	含义
network 输出	连接的状态，供下面的调用使用。
client 输入输出	客户端，已由 mqtt_client_init(§3.5.1)初始化。
host 输入	要连接的主机，以名字或地址表示。
port 输入	端口，以文本表示，例如 "1883"。
tls 输入	使用什么 TLS(表 3.7)。
返回	MQTT_OK、MQTT_CONNECTION_CLOSED 或 MQTT_TLS_ERROR。

api-reference-zh.tex Page 63 November 8, 2026 13:47

表 3.7 mqtt_net_client_tls 的字段

字段	含义
enabled	true 表示使用 TLS。只有这时才读取其余字段。
ca_file	要信任的证书。
insecure	没有 ca_file 时，true 表示不检查服务器的证书。这样一来，两者之间的任何人都可以读取和篡改一切。
server_name	服务器证书中应有的名字，NULL 表示用 host。
certificate_file、key_file、key_password	客户端的证书链、它的密钥和密钥的口令，或 NULL。

在一个程序中证书只加载一次：之后的连接沿用第一次加载的。

3.8.2 mqtt_net_client_step

搬运能搬运的：运行定时器，读取到达的内容（这会引发事件），写出等待发送的内容；如果什么也没有，就最多等待给定的时间。对带 platform/ 存储区的客户端，它在写出之前封好并同步所收内容的记录，好让等待它们的应答发出；有记录还没有封组时，它不等待。

```
mqtt_status mqtt_net_client_step(mqtt_net_client *network, uint32_t wait_ms);
```

见 *include/base_net.h*

参数	含义
network 输入输出	客户端的网络连接。
wait_ms 输入	最长的等待时间，以毫秒计。
返回	MQTT_OK；连接已经断开或者应当关闭后为 MQTT_CONNECTION_CLOSED：此时调用 mqtt_net_client_close（§3.8.3）。

api-reference-zh.tex Page 64 November 8, 2026 13:47

3.8.3 mqtt_net_client_close

关闭客户端的网络连接，并用 mqtt_client_connection_lost（§3.6.5）把这件事报告给客户端。有存储区的客户端会先封好并同步它最后的记录。

```
void mqtt_net_client_close(mqtt_net_client *network);
```

见 *include/base_net.h*

参数	含义
network 输入输出	客户端的网络连接。
返回	无。

3.9 在程序自己的循环中运行

自己搬运字节的程序打开连接，调用 mqtt_client_connect（§3.6.1），然后在每一轮中：

1. 调用 mqtt_client_tick（§3.9.7），它运行定时器，并说明可以等待多久。
2. 写出 mqtt_client_transmit_view（§3.9.5）的各段，并以套接字接收的字节数调用 mqtt_client_transmit_consume（§3.9.6）。
3. 读入 mqtt_client_receive_space（§3.9.2）给出的位置，并以读到的字节数调用 mqtt_client_receive_commit（§3.9.3）。事件在那次调用期间到来。
4. 有存储区时，用 mqtt_client_store_commit（§3.11.2）封好这一轮的记录，保存它们，再用 mqtt_client_stored（§3.11.3）报告（第 3.11 节）。
5. 当 mqtt_client_should_close（§3.6.4）这样说，或者套接字出错时，关闭连接，并调用 mqtt_client_connection_lost（§3.6.5）。

api-reference-zh.tex Page 65 November 8, 2026 13:47

3.9.1 mqtt_client_receive_capacity

求出现在可以交给客户端多少字节。

```
uint32_t mqtt_client_receive_capacity(const mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入	客户端。
返回	字节数。

3.9.2 mqtt_client_receive_space

求出来自服务器的下一批字节可以放在哪里：接收缓冲区的剩余部分，以给回复留出空间为限。

```
uint32_t mqtt_client_receive_space(const mqtt_client *client,
                                   mqtt_space *space);
```

见 *include/base_client.h*

参数	含义
client 输入	客户端。
space 输出	位置。
返回	它能容纳的字节数；现在没有空间时为零。

api-reference-zh.tex Page 66 November 8, 2026 13:47

3.9.3 mqtt_client_receive_commit

接收写入 mqtt_client_receive_space(§3.9.2)所给位置的字节。每个完整的报文就地处理,这会引发它的事件;只到达了一部分的报文,它的开头会被移到缓冲区的前端。

```
mqtt_status mqtt_client_receive_commit(mqtt_client *client, uint32_t count,
                                       uint64_t now_ms);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
count 输入	写入的字节数。超出该位置容量的部分不计。
now_ms 输入	时间。
返回	MQTT_OK, 或者连接现在正在关闭的原因。

api-reference-zh.tex Page 67 November 8, 2026 13:47

3.9.4 mqtt_client_receive

从程序的缓冲区接收到达的字节。

```
mqtt_status mqtt_client_receive(mqtt_client *client, const uint8_t *data,
                                uint32_t length, uint64_t now_ms);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
data 输入	字节。
length 输入	data 中的字节数。
now_ms 输入	时间。
返回	MQTT_OK, 或者连接现在正在关闭的原因。

api-reference-zh.tex Page 68 November 8, 2026 13:47

3.9.5 mqtt_client_transmit_view

求出接下来要发送的字节，以一次 writev 可以一起发送的若干段给出。

```
uint32_t mqtt_client_transmit_view(mqtt_client *client, mqtt_run *runs,
                                   uint32_t max_runs);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
runs 输出	这些段，按发送顺序排列，在有关这个客户端的下一次调用之前有效。
max_runs 输入	runs 能容纳的项数。环的每一块一项，再加一项，就足以容纳所有等待发送的内容；为一时只给出第一段。
返回	段的数目；现在没有要发送的时为零。

3.9.6 mqtt_client_transmit_consume

报告视图中有多少字节已经发出，从它的第一段算起。

```
void mqtt_client_transmit_consume(mqtt_client *client, uint32_t count);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
count 输入	发出的字节数，可以在任何一段中结束。超出等待发送的部分不计。
返回	无。

api-reference-zh.tex Page 69 November 8, 2026 13:47

3.9.7 mqtt_client_tick

运行定时器：Keep Alive（到期时发送 PINGREQ），以及对 CONNACK 的等待。

```
uint32_t mqtt_client_tick(mqtt_client *client, uint64_t now_ms);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
now_ms 输入	时间。
返回	距离它应当再次运行的毫秒数。

3.10 让会话状态在重启后仍在

客户端可以像服务器一样把会话状态保存在存储区中（第 4.5 节）：它是否有会话、用的是哪个版本和客户端标识符、它为服务器暂存的消息、它收到的 QoS 2 消息的报文标识符，以及在 MQTT 3.1 中等待回应的 SUBSCRIBE 和 UNSUBSCRIBE 报文。在同一个存储区上再次启动时，它处于离线状态，带着原来的会话：它暂存的东西在下一个继续该会话的连接上发出，已经发送过的消息带上 DUP。

一旦客户端保存存储区，告诉服务器某样东西已被保存的应答，就要等它所在的组存储之后才发出：PUBACK、PUBREC、PUBREL、PUBCOMP、QoS 2 的 PUBLISH、CONNECT，以及在 MQTT 3.1 中的 SUBSCRIBE 和 UNSUBSCRIBE。来自服务器的消息在对它的应答发出之前就到达事件回调，所以在那里保存所得内容的程序不会丢失任何东西。

api-reference-zh.tex Page 70 November 8, 2026 13:47

表 3.8 mqtt_client_store_config 的字段

字段	含义
append	把日志的每条记录交给程序保存。
user_tag	交给 append 的值。存储中的 user_tag 则交给事件。
sessions	会话的记录必须经受得住什么(表 4.4):它的版本、它是否存在,以及客户端标识符。MQTT_STORE_MEMORY 什么都不写。
messages	消息的记录必须经受得住什么:为服务器暂存的消息、收到的 QoS 2 报文标识符,以及在 MQTT 3.1 中等待回应的 SUBSCRIBE 和 UNSUBSCRIBE 报文。MQTT_STORE_MEMORY 保留会话,但不保留它的消息。
snapshot_bytes	日志写到多少字节后应做一次快照。

没有什么会用默认值填写它: mqtt_client_init (§3.5.1) 让客户端不带存储区,而 mqtt_net_client_store_config (§3.10.2) 为 platform/ 的存储区填写它。使用 platform/ 时,程序初始化客户端,用 mqtt_platform_store_open(§4.5.2)打开存储区,填写配置,用 mqtt_client_store_init(§3.10.1)交给客户端,再用 mqtt_net_client_restore (§3.10.3) 放回存储区保存的东西,这些都在第一个连接之前。此后每次 mqtt_net_client_step(§3.8.2)完成其余的事。程序同一时间只运行一个带 platform/ 存储区的客户端。

api-reference-zh.tex Page 71 November 8, 2026 13:47

3.10.1 mqtt_client_store_init

给客户端一个存储区，在 mqtt_client_init（§3.5.1）之后、连接或发布任何东西之前调用。然后用 mqtt_net_client_restore（§3.10.3）或 mqtt_client_restore（§3.11.1）放回存储区保存的东西。在第一次快照之前什么都不写，而第一次快照必须在第一个连接之前完整。

```
mqtt_status mqtt_client_store_init(mqtt_client *client,
                                   const mqtt_client_store_config *config);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
config 输入	配置（表 3.8），会被复制。
返回	MQTT_OK；客户端没有初始化或配置不对时为 MQTT_INVALID_ARGUMENT；客户端已经用过，或者已经有存储区时为 MQTT_INVALID_STATE。

api-reference-zh.tex Page 72 November 8, 2026 13:47

3.10.2 mqtt_net_client_store_config

为 platform/ 的存储区填写客户端的存储区配置，供 mqtt_client_store_init（§3.10.1）使用。记录交给 mqtt_platform_store_append，它得到的标签就是存储区。会话和消息都按经受得住断电的要求保存，程序可以在这次调用之后把它降为 MQTT_STORE_WRITTEN；日志写到 1 MiB 后，或者写到存储区一个位置的一半（取较小者）后，就应做一次快照。

```
void mqtt_net_client_store_config(mqtt_client_store_config *config,
                                  int32_t store);
```

见 *include/base_net.h*

参数	含义
config 输出	配置。
store 输入	存储区，来自 mqtt_platform_store_open（§4.5.2）。
返回	无。

api-reference-zh.tex Page 73 November 8, 2026 13:47

3.10.3 mqtt_net_client_restore

把存储区保存的东西放回刚用 mqtt_client_store_init（§3.10.1）得到存储区的客户端，并在第一个连接之前做第一次快照。损坏的、来自更新版本的，或者超出这个客户端容量的存储区，会原封不动地移到一旁，控制台上有一行说明原因，客户端从空状态启动。此后每次 mqtt_net_client_step（§3.8.2）都封好并同步客户端的记录，让等待它们的应答发出。

```
mqtt_status mqtt_net_client_restore(mqtt_client *client, int32_t store,
                                    mqtt_store_fault *fault);
```

见 *include/base_net.h*

参数	含义
client 输入输出	客户端。
store 输入	存储区。
fault 输出	被移到一旁的存储区出了什么问题（表 4.7），或者 MQTT_STORE_FAULT_NONE。
返回	客户端可以连接（已恢复或为空）时为 MQTT_OK；按 mqtt_client_restore（§3.11.1）的结果给出 MQTT_INVALID_ARGUMENT 或 MQTT_INVALID_STATE；损坏的存储区无法移到一旁时为 MQTT_INVALID_STATE；或者第一次快照没能写出的原因。

和服务器一样，存储区被移到一旁之后返回的也是 MQTT_OK，只有 fault 能区分这两种情况。

api-reference-zh.tex Page 74 November 8, 2026 13:47

3.10.4 mqtt_client_store_identifier

给出存储区所保存的会话的客户端标识符：上一个 CONNECT 给出的那个，或者服务器分配的那个。这样，让服务器分配标识符的程序，可以在继续该会话的 CONNECT 中给出它。

```
mqtt_status mqtt_client_store_identifier(const mqtt_client *client,
                                         mqtt_utf8 *identifier);
```

见 *include/base_client.h*

参数	含义
client 输入	客户端。
identifier 输出	标识符，在下一个连接之前有效。
返回	MQTT_OK；不知道标识符时为 MQTT_NOT_FOUND。

3.11 在程序自己的循环中使用存储区

自己保存存储区的程序，填写它自己的一个 mqtt_client_store_config，再用 mqtt_client_store_init（§3.10.1）把它交给客户端。客户端的快照只包含它的内存中的东西，所以总是在两轮之间一次做完。然后：

1. 用 mqtt_client_restore（§3.11.1）放回存储区保存的东西。如果它给出 MQTT_STORE_DAMAGED，就把存储区移到一旁，让它重新从空开始。
2. 在第一个连接之前，以及每当 mqtt_client_snapshot_due（§3.11.6）这样说时，做一次完整的快照：调用 mqtt_client_snapshot_begin（§3.11.7），用快照的头部开始第一份副本，加入 mqtt_client_snapshot_next（§3.11.8）给出的每条记录，直到 MQTT_NOT_FOUND，然后同步它。如果 new_segment 为 true，再清空新一代的日志位置，并在那里写入日志的头部。调用 mqtt_client_snapshot_done（§3.11.9），然后写出其它副本。
3. 每一轮在收到的字节处理完之后，用 mqtt_client_store_commit（§3.11.2）封好它的记录，如果 COMMIT 要求就同步，然后用 mqtt_client_stored（§3.11.3）报告它们；同步失败时则调用 mqtt_client_store_failed（§3.11.4）。

api-reference-zh.tex Page 75 November 8, 2026 13:47

4. 当 mqtt_client_store_open(§3.11.5)说有记录还没有封组时，不要等待：先把它们封好。

3.11.1 mqtt_client_restore

把存储区保存的东西放回刚得到存储区的客户端。先读取并检查每个位置，然后应用其日志段能够接续的最新一份快照副本，再应用这些日志段，直到各自最后一个 COMMIT。每条记录都像服务器发来的内容一样受到检查，并且必须放得进这个客户端的内存。

```
mqtt_status mqtt_client_restore(mqtt_client *client,
                                const mqtt_store_source *source,
                                uint8_t *buffer, uint32_t size,
                                mqtt_store_fault *fault);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端，带存储区，并且还什么都没有。
source 输入	存储区，有 1 到 MQTT_CLIENT_RESTORE_COPIES(8)份副本(表 4.6)。
buffer 输出	读取用的空间，至少 MQTT_CLIENT_RESTORE_MINIMUM (256) 个字节。报文和客户端标识符直接读入页区。
size 输入	buffer 中的字节数。
fault 输出	结果为 MQTT_STORE_DAMAGED 时，说明出了什么问题。
返回	MQTT_OK；MQTT_STORE_DAMAGED；客户端没有初始化、不保存任何东西或某个参数不对时为 MQTT_INVALID_ARGUMENT；客户端已经有东西时为 MQTT_INVALID_STATE。

它要么全部完成，要么什么都不做。只要有任何问题，客户端就会带着它的存储区重新初始化为空，由程序把存储区移到一旁。什么都没有的存储区给出 MQTT_OK，并且不改变任何东西。

api-reference-zh.tex Page 76 November 8, 2026 13:47

3.11.2 mqtt_client_store_commit

把上次调用以来写下的记录封成一组，加上一条 COMMIT 记录。等待它们的应答，在程序用 mqtt_client_stored(§3.11.3)报告这一组已经存储后发出。循环在每一轮收到的字节处理完之后调用一次，好让一次同步覆盖它们引起的所有东西。

```
mqtt_status mqtt_client_store_commit(mqtt_client *client, uint64_t now_ms,
                                     uint64_t *sequence);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
now_ms 输入	时间。
sequence 输出	封好的最后一组，也就是要报告的那一组。
返回	MQTT_OK，没有什么可封时也是；客户端没有初始化或不保存任何东西时为 MQTT_INVALID_ARGUMENT；日志已断时为 MQTT_INVALID_STATE。

api-reference-zh.tex Page 77 November 8, 2026 13:47

3.11.3 mqtt_client_stored

报告直到某一组为止的每一组都已按其 COMMIT 的要求存储，好让等待它们的应答发出。

```
void mqtt_client_stored(mqtt_client *client, uint64_t sequence);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
sequence 输入	已存储的最后一组。
返回	无。

3.11.4 mqtt_client_store_failed

报告记录可能已经丢失，例如同步失败时。日志于是断开：不再写入任何东西，每个应答都要等待，直到新一代的快照完整为止。

```
void mqtt_client_store_failed(mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
返回	无。

api-reference-zh.tex Page 78 November 8, 2026 13:47

3.11.5 mqtt_client_store_open

查明是否写下了还没有 COMMIT 封好的记录。发现有的循环，在等待之前调用 mqtt_client_store_commit(§3.11.2)。

```
bool mqtt_client_store_open(const mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入	客户端。
返回	有时为 true。

3.11.6 mqtt_client_snapshot_due

查明是否应该开始一次快照：这一代还没有完整的快照，日志已断，或者日志段的长度已经超过了 snapshot_bytes 字节。

```
bool mqtt_client_snapshot_due(const mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入	客户端。
返回	应该开始、并且没有快照在进行中时为 true。

api-reference-zh.tex Page 79 November 8, 2026 13:47

3.11.7 mqtt_client_snapshot_begin

开始一次快照，它一次做完：在 mqtt_client_snapshot_done（§3.11.9）或 mqtt_client_snapshot_abort （§3.11.10） 之前， 不能以其它方式调用这个客户端。新一代的快照开始一个新的日志段， 程序在快照的第一份副本完整之后、 mqtt_client_snapshot_done（§3.11.9）之前，用日志的头部开始这个日志段。之前的每一组都必须已经用 mqtt_client_store_commit（§3.11.2）封好。

```
mqtt_status mqtt_client_snapshot_begin(mqtt_client *client, uint64_t now_ms,
                                       uint8_t *snapshot_header,
                                       uint8_t *journal_header,
                                       bool *new_segment);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
now_ms 输入	时间。
snapshot_header 输出	快照头部的 MQTT_STORE_HEADER_SIZE（58）个字节的空间。
journal_header 输出	日志段头部的 MQTT_STORE_HEADER_SIZE 个字节的空间。
new_segment 输出	开始一个新的日志段时为 true。
返回	MQTT_OK；客户端没有初始化或不保存任何东西时为 MQTT_INVALID_ARGUMENT；有组还没封好或已有快照在进行中时为 MQTT_INVALID_STATE。

api-reference-zh.tex Page 80 November 8, 2026 13:47

3.11.8 mqtt_client_snapshot_next

给出进行中的快照的下一条记录。

```
mqtt_status mqtt_client_snapshot_next(mqtt_client *client, uint8_t *space,
                                      uint32_t size,
                                      mqtt_store_record *record);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
space 输出	记录的帧和固定字段放在这里，在下一次调用之前有效。
size 输入	space 中的字节数，至少 MQTT_CLIENT_SNAPSHOT_SPACE(32)。
record 输出	记录(表 4.5)。它的各段在下一次调用之前有效。
返回	有记录时为 MQTT_OK；最后一条已经给出后为 MQTT_NOT_FOUND；space 太小时为 MQTT_BUFFER_TOO_SMALL；没有快照在进行中，或者客户端在快照开始后有了改动时，为 MQTT_INVALID_STATE。

3.11.9 mqtt_client_snapshot_done

报告快照的第一份副本已经完整，它要开始的日志段也已开始，于是下一次快照开始新的一代。日志中出现断档之后，等待这次快照的应答，在它们的组存储之后发出。

```
void mqtt_client_snapshot_done(mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
返回	无。

api-reference-zh.tex Page 81 November 8, 2026 13:47

3.11.10 mqtt_client_snapshot_abort

放弃进行中的快照。

```
void mqtt_client_snapshot_abort(mqtt_client *client);
```

见 *include/base_client.h*

参数	含义
client 输入输出	客户端。
返回	无。

4

两者共用

本章是服务器和客户端都使用的类型和函数：字节视图、MQTT 5.0 的属性、原因码、TLS、存储区、platform/ 为它所运行的系统上的程序提供的东西，以及库的版本。

4.1 字节视图

表 4.1 字节视图

类型	字段	用于
mqtt_utf8	length(16 位)、data	UTF-8 编码字符串：主题、客户端标识符、字符串类型的属性。
mqtt_binary	length(16 位)、data	二进制数据：密码、遗嘱有效载荷、Correlation Data。
mqtt_payload	length(32 位)、data	PUBLISH 的有效载荷。
mqtt_run	data、length	要写出的字节，来自 transmit_view。
mqtt_space	data、length	可读入的空间，来自 receive_space。
mqtt5_utf8_pair	name、value	一个用户属性。

它们都不拥有自己的字节，末尾也都没有 NUL。client_subscribe.c 是这样打印其中一个的：

```
printf("%.*s", (int)s.length, (const char *)s.data);
```

api-reference-zh.tex Page 84 November 8, 2026 13:47

4.1.1 mqtt_utf8_from_text

为一个以 NUL 结尾的字符串生成视图，不复制它。字符串必须比视图存在得更久。

```
mqtt_utf8 mqtt_utf8_from_text(const char *text);
```

见 *include/base_wire.h*

参数	含义
text 输入	文本，NULL 表示空视图。
返回	视图。

4.2 属性

MQTT 5.0 报文的属性是一个 mqtt5_properties：每个属性一个字段，以及一个说明哪些属性存在的掩码。只有标记为存在的属性才会被发送，或者在读取时有值。5.0 之前，报文没有属性，它们会被忽略。

- 发送一个。用 mqtt5_properties_clear（§4.2.1）清空这一组（或者从 mqtt_connect_clear（§4.3.2）、mqtt_reason_clear（§4.3.1）或一个清零的报文开始），设置字段，再用 mqtt5_properties_mark（§4.2.2）标记它。用户属性是 user_pairs 和 user_pair_count 中的一个键值对数组，不用标记。
- 读取一个。先问 mqtt5_properties_has（§4.2.3），再读取字段。
- 读取每一个。可以出现不止一次的属性，例如用户属性，要用读取器从 raw 中读取，raw 是原样到达的整个属性块：对 raw 调用 mqtt_reader_init，然后调用 mqtt5_property_next（§4.2.4），直到它返回 false。

```
mqtt5_properties_clear(&properties);
properties.message_expiry_interval = 60U;   /* 秒 */
mqtt5_properties_mark(&properties, MQTT5_PROP_MESSAGE_EXPIRY_INTERVAL);

if (mqtt5_properties_has(&event->publish->properties, MQTT5_PROP_CONTENT_TYPE)) {
    mqtt_utf8 type = event->publish->properties.content_type;
}
```

api-reference-zh.tex Page 85 November 8, 2026 13:47

表 4.2 各个属性, 以及每个属性的字段和它在 v5_prop.h 中的标识符

字段	标识符
payload_format_indicator	MQTT5_PROP_PAYLOAD_FORMAT_INDICATOR
message_expiry_interval	MQTT5_PROP_MESSAGE_EXPIRY_INTERVAL
content_type	MQTT5_PROP_CONTENT_TYPE
response_topic	MQTT5_PROP_RESPONSE_TOPIC
correlation_data	MQTT5_PROP_CORRELATION_DATA
subscription_identifier	MQTT5_PROP_SUBSCRIPTION_IDENTIFIER: 有多个时取第一个
session_expiry_interval	MQTT5_PROP_SESSION_EXPIRY_INTERVAL
assigned_client_identifier	MQTT5_PROP_ASSIGNED_CLIENT_IDENTIFIER
server_keep_alive	MQTT5_PROP_SERVER_KEEP_ALIVE
authentication_method	MQTT5_PROP_AUTHENTICATION_METHOD
authentication_data	MQTT5_PROP_AUTHENTICATION_DATA
request_problem_information	MQTT5_PROP_REQUEST_PROBLEM_INFORMATION
will_delay_interval	MQTT5_PROP_WILL_DELAY_INTERVAL
request_response_information	MQTT5_PROP_REQUEST_RESPONSE_INFORMATION
response_information	MQTT5_PROP_RESPONSE_INFORMATION
server_reference	MQTT5_PROP_SERVER_REFERENCE
reason_string	MQTT5_PROP_REASON_STRING
receive_maximum	MQTT5_PROP_RECEIVE_MAXIMUM
topic_alias_maximum	MQTT5_PROP_TOPIC_ALIAS_MAXIMUM
topic_alias	MQTT5_PROP_TOPIC_ALIAS
maximum_qos	MQTT5_PROP_MAXIMUM_QOS
retain_available	MQTT5_PROP_RETAIN_AVAILABLE
user_pairs、user_pair_count	MQTT5_PROP_USER_PROPERTY: 设置键值对, 不用标记
maximum_packet_size	MQTT5_PROP_MAXIMUM_PACKET_SIZE

续页

api-reference-zh.tex Page 86 November 8, 2026 13:47

表 4.2 各个属性(续)

字段	标识符
wildcard_subscription_available	MQTT5_PROP_WILDCARD_SUBSCRIPTION_AVAILABLE
subscription_identifier_available	MQTT5_PROP_SUBSCRIPTION_IDENTIFIER_AVAILABLE
shared_subscription_available	MQTT5_PROP_SHARED_SUBSCRIPTION_AVAILABLE

4.2.1 mqtt5_properties_clear

清空一组属性: 不把任何属性标记为存在, 并丢弃 raw 和用户属性。其它字段保留原有的内容, 因为在被标记之前, 它们都不会被读取。

```
void mqtt5_properties_clear(mqtt5_properties *properties);
```

见 *include/v5_properties.h*

参数	含义
properties 输出	属性。
返回	无。

api-reference-zh.tex Page 87 November 8, 2026 13:47

4.2.2 mqtt5_properties_mark

在设置了某个属性的字段之后，把它标记为存在。

```
void mqtt5_properties_mark(mqtt5_properties *properties, uint32_t identifier);
```

见 *include/v5_properties.h*

参数	含义
properties 输入输出	属性。
identifier 输入	它的标识符，来自 v5_prop.h。
返回	无。

4.2.3 mqtt5_properties_has

求出某个属性是否存在。

```
bool mqtt5_properties_has(const mqtt5_properties *properties,
                          uint32_t identifier);
```

见 *include/v5_properties.h*

参数	含义
properties 输入	属性。
identifier 输入	它的标识符，来自 v5_prop.h。
返回	存在时为 true。

api-reference-zh.tex Page 88 November 8, 2026 13:47

4.2.4 mqtt5_property_next

读取一个已收到、并且已经检查过的属性块中的下一个属性：每个用户属性和每个订阅标识符都单独给出。

```
bool mqtt5_property_next(mqtt_reader *cursor, mqtt5_property *property);
```

见 *include/v5_properties.h*

参数	含义
cursor 输入输出	一个针对 raw 的读取器，用 mqtt_reader_init(\&cursor, raw.data, raw.length) 开始，该函数来自 base_wire.h。
property 输出	属性：它的 identifier，以及它在 integer、text 或 binary 中的值，对用户属性则是 text 和 value。
返回	有一个时为 true，到了属性块的末尾为 false。

4.3 原因码

DISCONNECT 和 AUTH 报文用 mqtt_reason 表示，其中是原因码 reason_code 和属性 properties。MQTT 5.0 的原因码是 v5_rc.h 中以 MQTT5_RC_ 开头的宏，3.1.1 和 3.1 的返回码则在 v311_rc.h 和 v31_rc.h 中。

4.3.1 mqtt_reason_clear

清空一个 DISCONNECT 或 AUTH：原因码为 0，没有属性。

```
void mqtt_reason_clear(mqtt_reason *packet);
```

见 *include/base_packet.h*

参数	含义
packet 输出	报文。
返回	无。

api-reference-zh.tex Page 89 November 8, 2026 13:47

4.3.2 mqtt_connect_clear

清空一个 CONNECT：版本 3.1.1，每个标志为 false，Keep Alive 为 0，每个字符串为空，没有属性。它比把整个结构清零开销更小，而且这块内存事先可以是任何内容。

```
void mqtt_connect_clear(mqtt_connect *packet);
```

见 *include/base_packet.h*

参数	含义
packet 输出	报文。
返回	无。

4.3.3 mqtt5_reason_code

求出报告某个 mqtt_status 的原因码，服务器的回调可能需要这样做。

```
uint8_t mqtt5_reason_code(mqtt_status status);
```

见 *include/base_packet.h*

参数	含义
status 输入	一个不是 MQTT_OK 的状态。
返回	原因码；没有更接近的时为 MQTT5_RC_UNSPECIFIED_ERROR。

4.4 TLS

TLS 在 platform/ 中，经由 Mbed TLS 实现，存在于用 WITH_TLS=yes 构建的库里。platform/ 的循环自己使用它：服务器通过它的选项中的 TLS 字段，客户端通过 mqtt_net_client_tls。程序自己的循环也可以使用同样的函数，表 4.3 列出了它们。

api-reference-zh.tex Page 90 November 8, 2026 13:47

4.4.1 mqtt_tls_available

求出库构建时是否带有 TLS。不带 TLS 构建时，base_tls.h 的其它每个函数都报告 MQTT_TLS_ERROR。

```
bool mqtt_tls_available(void);
```

见 *include/base_tls.h*

参数	含义
无参数。	
返回	带有时为 true。

表 4.3 base_tls.h 的函数

函数	作用
mqtt_tls_server_setup	加载服务器的证书链和密钥(PEM 格式)，以及用来检查客户端的证书(如果有)。
mqtt_tls_client_setup	加载客户端信任的证书和它自己的证书，在一个程序中只加载一次。
mqtt_tls_session_open	在已连接的套接字上开始一个 TLS 会话，任何一侧都可以用。没有空间再容纳一个时给出 MQTT_TLS_NONE。
mqtt_tls_read、mqtt_tls_write	经由一个 TLS 会话读和写，就像 mqtt_platform_read 和 mqtt_platform_write 在套接字上做的那样。
mqtt_tls_wants_read、mqtt_tls_wants_write	说明一个 TLS 会话要继续下去，是否需要套接字可读或可写。
mqtt_tls_has_pending	说明一个 TLS 会话是否持有已经从套接字读出的字节，等待套接字是看不出这些字节的。
mqtt_tls_session_close	结束一个 TLS 会话并释放它。

4.5 存储区

存储区保存服务器或客户端在重启时不能丢失的东西，形式是记录，其含义由库决定：程序或 platform/ 只负责保存它们的字节。base_store.h 定义了这些记录。存储区有两部

api-reference-zh.tex Page 91 November 8, 2026 13:47

分。快照以记录的形式保存整个状态，保存为两份或更多副本，依次写出。日志保存快照之后的每个改动，存放在两个轮流使用的位置中，快照的每一代用一个。每份副本和每个日志段都以一个 MQTT_STORE_HEADER_SIZE(58)字节的头部开始，每条记录都带有 CRC-16，日志由一系列组构成，每组以一条 COMMIT 记录结束。恢复时，一组要么整个采用，要么完全不用，所以被崩溃截断的写入只会丢失它所在的那一组，而那一组的应答从未发出。CRC 能发现损坏，却发现不了伪造，所以存储区应当放在只有该程序能写入的地方。

表 4.4 mqtt_store_durability 的取值

值	记录在依赖它的东西发出之前经受得住什么
MQTT_STORE_MEMORY	什么都不经受：根本不写。
MQTT_STORE_WRITTEN	进程崩溃：写入介质。
MQTT_STORE_SYNCED	断电：写入，并且同步。

服务器的 store_append 和客户端的 append 都是这个类型。记录(表 4.5)的字节只在调用期间有效，所以要在它返回之前写出或复制。它返回 MQTT_OK；记录无法加入时返回其它任何值，这会让日志断开，直到下一次快照完整为止。

```
typedef mqtt_status (*mqtt_store_append)(uint32_t user_tag,
                                         const mqtt_store_record *record);
```

见 *include/base_store.h*

表 4.5 mqtt_store_record 的字段

字段	含义
runs、run_count	记录的字节，包括它的帧和 CRC，按顺序排列，最多分成 MQTT_STORE_RECORD_RUNS(6)段。
length	所有段的字节数。
sequence	记录所属的组。
durability	它必须经受得住什么。对 COMMIT 来说，是它那一组中任何记录要求的最高程度：如果是 MQTT_STORE_SYNCED，这一组就需要同步。
kind	记录的种类。COMMIT 是 MQTT_STORE_RECORD_COMMIT。

恢复通过一个 mqtt_store_source(表 4.6)读取存储区，其中的函数是这个类型。它

api-reference-zh.tex Page 92 November 8, 2026 13:47

从某一部分(MQTT_STORE_SNAPSHOT 或 MQTT_STORE_JOURNAL)的一个位置读取:快照的一份副本,或者日志的一个位置。count 只在位置的末尾才小于 capacity,超过末尾或位置不存在时为 0。它返回 MQTT_OK;介质无法读取时返回其它任何值。

```
typedef mqtt_status (*mqtt_store_read)(uint32_t user_tag, mqtt_store_part part,
                                       uint32_t place, uint64_t offset,
                                       uint8_t *data, uint32_t capacity,
                                       uint32_t *count);
```

见 *include/base_store.h*

表 4.6 mqtt_store_source 的字段

字段	含义
read、user_tag	读取存储区的函数,以及交给它的值。
copies	快照的副本数。
erased	从未写过的字节的值:文件中为 0x00,闪存中为 0xFF。没有通过检查的记录,只有在它之后的每个字节都是这个值时,才算被崩溃截断;否则存储区就是损坏的。
downtime_ms	从最后一次写入到恢复的毫秒数;介质说不出时为 0。每个间隔剩下的部分恢复时都要减去它。

恢复发现存储区损坏,或者不是它能接受的存储区时,会在一个 mqtt_store_fault 中说明问题出在哪里、是什么(表 4.7 和表 4.8)。

表 4.7 mqtt_store_fault 的字段

字段	含义
kind	出了什么问题(表 4.8)。
part、place	在哪里:存储区的哪一部分,以及哪份副本或哪个日志位置。
generation	副本或日志段的代;没有读到它的头部时为 0。
offset、record_kind	记录在它的位置中的偏移量和它的种类;对头部来说都是 0。

api-reference-zh.tex Page 93 November 8, 2026 13:47

表 4.8 mqtt_store_fault_kind 的取值

值	含义
MQTT_STORE_FAULT_NONE	没有问题。
MQTT_STORE_FAULT_NO_COPY	快照没有一份副本通过检查。
MQTT_STORE_FAULT_HEADER	某个日志段或副本的头部已损坏，或者不是这个库写的。
MQTT_STORE_FAULT_VERSION	某个日志段或副本的格式主版本更新。
MQTT_STORE_FAULT_RECORD	某条记录没有通过检查，而且它后面还有字节。
MQTT_STORE_FAULT_CONTENT	某条记录通过了检查，但其中有不该有的东西。
MQTT_STORE_FAULT_CHAIN	各日志段没有接续快照，或者彼此没有接续。
MQTT_STORE_FAULT_ROOM	给的内存太小，放不下保存的东西。
MQTT_STORE_FAULT_MEDIUM	介质无法读取。
MQTT_STORE_FAULT_CONFIG	某条记录要求配置不再提供的东西，或者超出了配置的某个上限。

4.5.1 mqtt_store_fault_name

为报告给出一种问题的名称。

```
const char *mqtt_store_fault_name(mqtt_store_fault_kind kind);
```

见 *include/base_store.h*

参数	含义
kind 输入	问题的种类。
返回	描述它的英文文字，例如 MQTT_STORE_FAULT_NONE 为 no fault。

在 FreeRTOS 和 uC/OS-III 上，platform/ 把存储区保存在一块闪存区域中，这块区域由板子命名并提供访问它的调用（见《构建 libterae》）；在其它每个有文件的系统上，则保存在一个文件夹的文件中。同一时间最多可以打开 MQTT_PLATFORM_STORES（2）个存储区：服务器一个，客户端一个。

api-reference-zh.tex Page 94 November 8, 2026 13:47

4.5.2 mqtt_platform_store_open

打开一个存储区，没有时就创建它。在文件夹中（文件夹不存在时会被创建），存储区就是这些文件：snapshot.0 起每份副本一个，以及 journal.0 和 journal.1；在 POSIX 系统上只有它们的所有者能够读写。在闪存中，区域被分成由整个擦除块组成的位置，每份副本一个，另外两个给日志。

```
mqtt_status mqtt_platform_store_open(const char *where, uint32_t copies,
                                     int32_t *store);
```

见 *include/base_platform.h*

参数	含义
where 输入	它的文件所在的文件夹，或者板子上区域的名字。
copies 输入	快照的副本数，从 MQTT_PLATFORM_STORE_COPIES_MIN（2）到 MQTT_PLATFORM_STORE_COPIES_MAX（8）。
store 输出	存储区；没有打开时为 MQTT_STORE_NONE。
返回	MQTT_OK；where 为空或太长，或者 copies 超出范围时为 MQTT_INVALID_ARGUMENT；已经打开了 MQTT_PLATFORM_STORES 个时为 MQTT_QUOTA_EXCEEDED；介质无法打开，或者系统没有介质时为 MQTT_INVALID_STATE。

4.5.3 mqtt_platform_store_close

关闭一个存储区。没有打开的存储区（包括 MQTT_STORE_NONE）会被忽略，所以不论之前发生了什么，程序都可以关闭它的存储区。

```
void mqtt_platform_store_close(int32_t store);
```

见 *include/base_platform.h*

参数	含义
store 输入	存储区。
返回	无。

api-reference-zh.tex Page 95 November 8, 2026 13:47

platform/ 的循环用表 4.9 中的其它调用保存它们的存储区。有自己循环的程序也可以使用它们，把存储区保存在 platform/ 会保存的地方。

表 4.9 base_platform.h 的其它存储区调用

函数	作用
mqtt_platform_store_append	一个 store_append，标签是存储区：把一条记录加到正在使用的日志位置，暂不同步。
mqtt_platform_store_commit	如果上次调用以来加入的某个 COMMIT 要求同步，就同步日志。
mqtt_platform_store_read	mqtt_store_source 的 read，标签是存储区。
mqtt_platform_store_segment	开始一个日志段：清空它这一代的位置，写入它的头部，然后同步。
mqtt_platform_store_copy_begin、mqtt_platform_store_copy_record、mqtt_platform_store_copy_end	写出快照的第一份副本：它的头部、它的记录，然后同步。
mqtt_platform_store_copy_duplicate	从第一份副本写出其它每份副本，每份同步之后再写下一份。失败之后可以再次调用。
mqtt_platform_store_set_aside	把存储区保存的一切以它自己的编号移到一旁，再让存储区从空开始。什么都不删除：在文件夹中，文件移到其中的文件夹 aside.1、aside.2 等等。
mqtt_platform_store_copies、mqtt_platform_store_erased、 mqtt_platform_store_capacity、mqtt_platform_store_downtime	mqtt_store_source 和快照需要的东西：副本数、从未写过的字节的值、一个位置最多能容纳多少，以及日志最后一次写入是多久以前。闪存说不出最后这一项，所以在板子上，程序停止期间各种间隔不会流逝。
mqtt_platform_parse_durability	把 memory、written 或 synced 读作一个持久程度。
mqtt_platform_store_remove	删除一个没有打开的存储区，供测试使用：程序只把存储区移到一旁，从不删除它。

在自己的介质上保存存储区的程序，只需复制它得到的字节，用不到 base_store.h 的其余部分（表 4.10）：那是库给记录加帧、检查和读取记录的方式。

api-reference-zh.tex Page 96 November 8, 2026 13:47

表 4.10 base_store.h 的其余部分

函数	作用
mqtt_store_crc16	每个头部和每条记录的 CRC-16：多项式 0x1021，不反射，没有最终异或，从 MQTT_STORE_CRC_INIT 开始。
mqtt_store_header_encode、mqtt_store_header_decode	写出和检查每份副本、每个日志段开头的头部。主版本高于 MQTT_STORE_FORMAT_MAJOR（2）的不读。
mqtt_store_frame_head、mqtt_store_frame_check、mqtt_store_frame_crc、mqtt_store_frame_tail	写出和检查记录的帧：它的种类和长度，带有它们自己的 CRC，以及内容之后的 CRC。
mqtt_store_put_u64、mqtt_store_get_u64	按网络字节序排列的八个字节。
mqtt_store_scan_place	读取并检查存储区的一个位置，就像恢复首先做的那样。

4.6 系统

base_platform.h 声明库需要从系统得到的东西，由 platform/ 的每个文件夹提供。程序也可以使用它，这样在时钟、随机字节或停止方面就不需要自己的东西。

4.6.1 mqtt_platform_time_ms

读取时钟，用于每个 now_ms。

```
uint64_t mqtt_platform_time_ms(void);
```

见 *include/base_platform.h*

参数	含义
无参数。	
返回	从某个任意时刻起的毫秒数。它只会向前走。

api-reference-zh.tex Page 97 November 8, 2026 13:47

4.6.2 mqtt_platform_random_bytes

从系统的随机数来源填充字节，攻击者无法预测它们：服务器的 random_bytes 用的就是这个来源。

```
mqtt_status mqtt_platform_random_bytes(uint8_t *data, uint32_t length);
```

见 *include/base_platform.h*

参数	含义
data 输出	放它们的地方。
length 输入	字节数。
返回	MQTT_OK；系统没有这样的来源或者它失败时为 MQTT_INVALID_STATE，这时 data 没有意义。

4.6.3 mqtt_platform_stop_requested

求出程序是否被要求停止，要求来自 mqtt_platform_request_stop（§4.6.4），或者在 POSIX 系统上来自信号。在 POSIX 系统上，第一次调用会开始监视 SIGINT 和 SIGTERM。

```
bool mqtt_platform_stop_requested(void);
```

见 *include/base_platform.h*

参数	含义
无参数。	
返回	被要求过时为 true。

api-reference-zh.tex Page 98 November 8, 2026 13:47

4.6.4 mqtt_platform_request_stop

要求程序停止， 就像信号那样： 用于没有信号的系统， 或者用于结束其它任务的任务。mqtt_net_server_run(§2.6.1)随后很快返回。

```
void mqtt_platform_request_stop(void);
```

见 *include/base_platform.h*

参数	含义
无参数。	
返回	无。

base_platform.h 的其余部分在表 4.11 中：套接字，用于程序自己的循环，就像 server_loop.c 使用的那样；以及 platform/ 的循环用来等待的东西。它的存储区调用在第 4.5 节。

表 4.11 base_platform.h 的其它函数

函数	作用
程序	
mqtt_platform_time_us	时钟, 以微秒计。
mqtt_platform_sleep_ms	休眠。
mqtt_platform_console_write	把文本写到诊断输出。
mqtt_platform_output_write	把字节写到正常输出。
mqtt_platform_page_size	系统页的大小, 用来与 MQTT_PAGE_SIZE 比较; 系统不提供时为 0。
mqtt_platform_parse_u32	从文本读取一个十进制数。
套接字, 从不阻塞	
mqtt_platform_listen	打开一个接受连接的 TCP 套接字。
mqtt_platform_accept	接受一个连接； 没有等待的连接时给出 MQTT_INCOMPLETE。
mqtt_platform_connect	打开一个 TCP 连接, 并等待它建立。

续页

api-reference-zh.tex Page 99 November 8, 2026 13:47

表 4.11 base_platform.h 的其它函数(续)

函数	作用
mqtt_platform_read、mqtt_platform_read_spaces	读入一个位置,或者在一次调用中读入几个位置。没有等待读取的内容时为 MQTT_INCOMPLETE。
mqtt_platform_write、mqtt_platform_write_runs	写出一段,或者在一次调用中写出几段。什么也没被接收时为 MQTT_INCOMPLETE。
mqtt_platform_send_buffer	设置套接字发送缓冲区的大小。
mqtt_platform_close	关闭一个套接字。
mqtt_platform_wait_socket	等待一个套接字。
分级等待,如服务器的循环所做的那样	
mqtt_platform_wait_start、mqtt_platform_wait_add、mqtt_platform_wait_interest、mqtt_platform_wait_remove、mqtt_platform_wait_tier、mqtt_platform_wait	维护一组按忙碌程度分级的套接字,并等待到期的那些。
mqtt_platform_wake_handle、mqtt_platform_wake_drain	停止时会变为可读的套接字,用来立即结束等待。
用于测试和基准测试	
mqtt_platform_socket_pair、mqtt_platform_close_resets、mqtt_platform_cpu_us、mqtt_platform_interrupt_after_s、mqtt_platform_fork、mqtt_platform_child_exit	没有这些功能的系统通过返回值说明这一点。

api-reference-zh.tex Page 100 November 8, 2026 13:47

4.7 库的版本

base_library.h 给出程序编译时所用头文件的版本，以及三个函数，说明程序运行时用的那个库是按什么构建的。与 libterae.a 链接的程序，两者总是相同。加载共享库的程序在启动时检查它，因为那个库可能是在别处构建的，版本不同，或页大小不同。版本由三个数组成，每个从 0 到 255：

表 4.12 版本，来自 base_library.h

宏	值	何时改变
MQTT_LIBRARY_VERSION_MAJOR	1	为上一版构建的程序可能要修改或重新构建。
MQTT_LIBRARY_VERSION_MINOR	0	库增加了东西，而为更早的次版本构建的程序仍然能用。
MQTT_LIBRARY_VERSION_PATCH	5	程序看得到的东西都没变。
MQTT_LIBRARY_VERSION	0x010005	三者任一改变时：每个占一个字节，主版本在前，所以更新的版本数值更大。

4.7.1 mqtt_library_compatible

求出用某个版本和页大小的头文件构建的程序能否使用该库：主版本相同，次版本不晚于库的次版本，页大小相同，因为程序按 MQTT_PAGE_SIZE 定下数组的大小，由库来填。修订版本不影响结果。

```
bool mqtt_library_compatible(uint32_t version, uint32_t page_size);
```

见 *include/base_library.h*

参数	含义
version 输入	程序构建时的 MQTT_LIBRARY_VERSION。
page_size 输入	程序构建时的 MQTT_PAGE_SIZE。
返回	能用时为 true。

api-reference-zh.tex Page 101 November 8, 2026 13:47

程序在做任何别的事之前，用它自己头文件里的数字来问：

```
if (!mqtt_library_compatible(MQTT_LIBRARY_VERSION, MQTT_PAGE_SIZE)) {
    mqtt_platform_console_write("libterae: another version or page size\n");
    return 1;
}
```

4.7.2 mqtt_library_version

求出库构建时的版本，用于报告。

```
uint32_t mqtt_library_version(void);
```

见 *include/base_library.h*

参数	含义
无参数。	
返回	它的 MQTT_LIBRARY_VERSION。

4.7.3 mqtt_library_page_size

求出库构建时的页大小。

```
uint32_t mqtt_library_page_size(void);
```

见 *include/base_library.h*

参数	含义
无参数。	
返回	它的 MQTT_PAGE_SIZE。

5

示例

这些程序就是按这里的样子，用 `-std=c99 -Wall -Wextra -Wpedantic -Werror` 针对库构建的，并且一起运行过：每个服务器，配一个订阅的客户端和一个发布三条消息的客户端，三条消息都由前者打印了出来。`server_store.c` 还被停止并重新启动了两次，一条等待离开了这么久的客户端的消息，最终到达了它。这里的注释译成了中文，代码与英文版完全相同。构建一个程序时，带上它的设置文件：

```
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U server_platform.c server_setup.c \
   build/libterae.a -o server_platform
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U server_store.c server_setup.c \
   build/libterae.a -o server_store
cc -std=c99 -Iinclude -DMQTT_PAGE_SIZE=16384U client_subscribe.c client_setup.c \
   build/libterae.a -o client_subscribe
```

5.1 服务器

`server_setup.c` 为服务器提供内存和配置，最多 64 个连接，`server_setup.h` 声明它。下面三个服务器都使用它。给它一个存储区时，它会设置配置，把服务器的会话状态保存在那里。

server_setup.h

```
/* 示例服务器共用的东西：它们的内存和配置。 */
#ifndef SERVER_SETUP_H
#define SERVER_SETUP_H

#include <stdint.h>

#include "base_server.h"

/* 连接的数目，也就是网络层所需的项数。 */
#define CONNECTIONS 64U
/* 服务器容纳的消息和投递，以及恢复时按存储区中的编号找到它们所需的项数：
```

api-reference-zh.tex Page 104 November 8, 2026 13:47


```
 * 两者之和。 */
#define MESSAGES        512U
#define DELIVERIES      1024U
#define RESTORE_ENTRIES (MESSAGES + DELIVERIES)

/* 给服务器提供内存和配置，并初始化它。给出 mqtt_platform_store_open 打开的
 * 存储区时，它把会话状态保存在那里；给出 MQTT_STORE_NONE 时，停止之后什么
 * 都不留下。 */
mqtt_status server_setup(mqtt_server *server, int32_t store);

#endif /* SERVER_SETUP_H */
```

server_setup.c

```
/* 示例服务器的内存和配置。 */
#include <stdint.h>

#include "base_net.h"
#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

/* 服务器得到的各种东西各有多少：这些数目就是它的上限。
 * CONNECTIONS、MESSAGES 和 DELIVERIES 在 server_setup.h 里，因为网络层和恢复
 * 也需要它们。 */
#define BLOCK_SIZE    1024U  /* 块池中一块的字节数 */
#define BLOCKS        512U   /* 多于 pool_borrow_reserve 与硬配额之和 */
#define SCRATCH       16384U
#define SESSIONS      64U
#define SUBSCRIPTIONS 256U
#define GROUPS        32U
#define NODES         256U
#define TEXT_PAGES    4U
#define ARENA_PAGES   64U
#define ALIASES       16U    /* 即配置中的 topic_alias_maximum */

/* 各个页区的页数，每个数目都向上取整为整页。 */
#define PAGES(count, per_page) (((count) + (per_page) - 1U) / (per_page))
#define META_PAGES                                                  \
    (PAGES(CONNECTIONS, MQTT_SERVER_PAGE_CONNECTIONS) +             \
     PAGES(SESSIONS, MQTT_SERVER_PAGE_SESSIONS) +                   \
     PAGES(DELIVERIES, MQTT_SERVER_PAGE_DELIVERIES) +               \
     PAGES(MESSAGES, MQTT_SERVER_PAGE_MESSAGES) +                   \
     PAGES(SUBSCRIPTIONS, MQTT_SERVER_PAGE_SUBSCRIPTIONS) +         \
     PAGES(GROUPS, MQTT_SERVER_PAGE_GROUPS) +                       \
     PAGES(NODES, MQTT_TREE_PAGE_NODES) + TEXT_PAGES + ARENA_PAGES)

static mqtt_connection connections[CONNECTIONS];
static uint8_t pool_bytes[BLOCKS * BLOCK_SIZE];
static uint32_t pool_next[BLOCKS];
static uint8_t scratch[SCRATCH];
static mqtt_topic_alias aliases[CONNECTIONS * ALIASES];
static mqtt_session sessions[SESSIONS];
static uint32_t session_table[SESSIONS];
static mqtt_delivery deliveries[DELIVERIES];
```

api-reference-zh.tex Page 105 November 8, 2026 13:47


```
static mqtt_message messages[MESSAGES];
static mqtt_subscription subscriptions[SUBSCRIPTIONS];
static mqtt_share_group groups[GROUPS];
static mqtt_tree_node nodes[NODES];
static mqtt_tree_branch branches[NODES];
static uint8_t topic_text[TEXT_PAGES * MQTT_PAGE_SIZE];
static uint8_t arena[ARENA_PAGES * MQTT_PAGE_SIZE];
static uint32_t page_meta[META_PAGES * MQTT_PAGE_META_WORDS];

/* 分配的客户端标识符，由系统的随机字节生成。 */
static mqtt_status random_bytes(uint32_t tag, uint8_t *data, uint32_t length)
{
    (void)tag;
    return mqtt_platform_random_bytes(data, length);
}

mqtt_status server_setup(mqtt_server *server, int32_t store)
{
    mqtt_server_config config;
    mqtt_server_storage storage;

    mqtt_server_config_default(&config);
    config.random_bytes = &random_bytes;
    config.topic_alias_maximum = ALIASES;
    if (store != MQTT_STORE_NONE) {
        /* 会话和消息经受得住断电，每个回调得到的标签都是这个存储区。 */
        mqtt_net_server_store_config(&config, store);
    }

    storage.connections = connections;
    storage.connection_count = CONNECTIONS;
    storage.pool_bytes = pool_bytes;
    storage.pool_next = pool_next;
    storage.pool_block_size = BLOCK_SIZE;
    storage.pool_block_count = BLOCKS;
    storage.receive_scratch = scratch;
    storage.receive_scratch_size = SCRATCH;
    storage.aliases = aliases;
    storage.sessions = sessions;
    storage.session_count = SESSIONS;
    storage.session_table = session_table;
    storage.session_table_count = SESSIONS;
    storage.deliveries = deliveries;
    storage.delivery_count = DELIVERIES;
    storage.messages = messages;
    storage.message_count = MESSAGES;
    storage.subscriptions = subscriptions;
    storage.subscription_count = SUBSCRIPTIONS;
    storage.groups = groups;
    storage.group_count = GROUPS;
    storage.nodes = nodes;
    storage.node_count = NODES;
    storage.branches = branches;
    storage.branch_count = NODES;
    storage.topic_text = topic_text;
    storage.topic_text_pages = TEXT_PAGES;
    storage.arena_bytes = arena;
```

api-reference-zh.tex Page 106 November 8, 2026 13:47


```
    storage.arena_pages = ARENA_PAGES;
    storage.page_meta = page_meta;
    storage.page_meta_words = META_PAGES * MQTT_PAGE_META_WORDS;

    return mqtt_server_init(server, &config, &storage);
}
```

server_platform.c 用 platform/ 运行它:整个程序就是选项加一次调用。

server_platform.c

```
/* TCP 端口 1883 上的服务器，由 platform/ 运行，直到 Ctrl-C。 */
#include <stdio.h>

#include "base_net.h"
#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

static mqtt_server server;
static mqtt_net_connection network[CONNECTIONS];

int main(void)
{
    mqtt_net_server_options options;
    mqtt_status status = server_setup(&server, MQTT_STORE_NONE);

    if (status == MQTT_OK) {
        options.host = NULL;             /* 每个本地地址 */
        options.port = "1883";           /* TCP 上的 MQTT */
        options.tls_port = NULL;         /* 不用 TLS */
        options.websocket_port = NULL;   /* 不用 WebSocket */
        options.certificate_file = NULL;
        options.key_file = NULL;
        options.key_password = NULL;
        options.ca_file = NULL;
        options.tier = NULL;             /* 默认的等待分级 */
        options.send_buffer = 0U;        /* 系统的发送缓冲区 */
        options.store = MQTT_STORE_NONE; /* 停止之后什么都不留下 */
        status = mqtt_net_server_run(&server, network, &options);
    }
    if (status != MQTT_OK) {
        fprintf(stderr, "the server stopped: status %d\n", (int)status);
    }
    return (status == MQTT_OK) ? 0 : 1;
}
```

server_store.c 与之相同，只是多了一个存储区，在文件夹 terae-store 中：停止后再次启动，它带着原来的会话继续（第 2.9 节）。

api-reference-zh.tex Page 107 November 8, 2026 13:47

server_store.c

```
/* TCP 端口 1883 上的服务器，由 platform/ 运行，直到 Ctrl-C。它把会话、会话的
 * 消息和保留消息保存在文件夹 terae-store 中：再次启动时，它从停下的地方
 * 继续。 */
#include <stdio.h>

#include "base_net.h"
#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

static mqtt_server server;
static mqtt_net_connection network[CONNECTIONS];
/* 供恢复按编号找到每条消息和每个投递的空间。 */
static mqtt_server_restore_entry map[RESTORE_ENTRIES];

int main(void)
{
    int32_t store = MQTT_STORE_NONE;
    mqtt_store_fault fault;
    mqtt_net_server_options options;
    /* 文件夹不存在时会被创建。快照有两份副本，所以一份正在写的时候，另一份
     * 总是完整的。 */
    mqtt_status status = mqtt_platform_store_open("terae-store", 2U, &store);

    if (status == MQTT_OK) {
        status = server_setup(&server, store);
    }
    if (status == MQTT_OK) {
        /* 存储区中的东西被放回来。损坏的存储区会被移到一旁，控制台上有一行
         * 说明，服务器从空状态启动。 */
        status = mqtt_net_server_restore(&server, store, map, RESTORE_ENTRIES,
                                         &fault);
    }
    if (status == MQTT_OK) {
        options.host = NULL;
        options.port = "1883";
        options.tls_port = NULL;
        options.websocket_port = NULL;
        options.certificate_file = NULL;
        options.key_file = NULL;
        options.key_password = NULL;
        options.ca_file = NULL;
        options.tier = NULL;
        options.send_buffer = 0U;
        options.store = store; /* 运行中写入、同步并复制 */
        status = mqtt_net_server_run(&server, network, &options);
    }
    mqtt_platform_store_close(store); /* 如果从未打开，就什么都不做 */
    if (status != MQTT_OK) {
        fprintf(stderr, "the server stopped: status %d\n", (int)status);
    }
    return (status == MQTT_OK) ? 0 : 1;
}
```

`server_loop.c` 在它自己的循环中运行它，使用 `base_platform.h` 的套接字，以展

api-reference-zh.tex Page 108 November 8, 2026 13:47

示 mqtt_net_server_run(§2.6.1)所做的那些调用。

server_loop.c

```
/* TCP 端口 1883 上的服务器，在它自己的循环里运行，直到 Ctrl-C。它每隔
 * 几毫秒查看每个套接字，这让它保持简短：真正的循环会等待就绪的套接字。 */
#include <stdint.h>
#include <stdio.h>

#include "base_platform.h"
#include "base_server.h"
#include "server_setup.h"

#define ROUND_MS 5U /* 两轮之间最长的休眠时间 */

static mqtt_server server;
static int32_t sockets[CONNECTIONS]; /* 按服务器的连接索引排列 */

static void close_connection(uint32_t c)
{
    mqtt_platform_close(sockets[c]);
    sockets[c] = MQTT_SOCKET_NONE;
    mqtt_server_connection_closed(&server, c, mqtt_platform_time_ms());
}

/* 在一个连接上搬运能搬运的：先收，后发。 */
static void serve(uint32_t c)
{
    mqtt_status status = MQTT_OK;

    if (mqtt_server_receive_wanted(&server, c)) {
        mqtt_space spaces[MQTT_PLATFORM_READ_SPACES];
        const uint32_t count = mqtt_server_receive_space(
            &server, c, spaces, MQTT_PLATFORM_READ_SPACES);
        uint32_t got = 0U;

        if (count > 0U) {
            status = mqtt_platform_read_spaces(sockets[c], spaces, count, &got);
            if (status != MQTT_OK) {
                got = 0U; /* MQTT_INCOMPLETE：什么也没来 */
            }
            /* 总在 receive_space 之后调用，即使什么也没有。 */
            (void)mqtt_server_receive_commit(&server, c, got,
                                             mqtt_platform_time_ms());
        }
    }
    if (status != MQTT_CONNECTION_CLOSED) {
        mqtt_run runs[MQTT_PLATFORM_WRITE_RUNS];
        const uint32_t count = mqtt_server_transmit_view(
            &server, c, runs, MQTT_PLATFORM_WRITE_RUNS);
        uint32_t taken = 0U;

        if (count > 0U) {
            status = mqtt_platform_write_runs(sockets[c], runs, count, &taken);
            if (status == MQTT_OK) {
                mqtt_server_transmit_consume(&server, c, taken);
            }
```

api-reference-zh.tex Page 109 November 8, 2026 13:47


```
        }
    }
    if ((status == MQTT_CONNECTION_CLOSED) ||
        mqtt_server_should_close(&server, c)) {
        close_connection(c);
    }
}

int main(void)
{
    int32_t listener = MQTT_SOCKET_NONE;
    int32_t handle = MQTT_SOCKET_NONE;
    uint32_t c = 0U;

    for (c = 0U; c < CONNECTIONS; c++) {
        sockets[c] = MQTT_SOCKET_NONE;
    }
    if ((server_setup(&server, MQTT_STORE_NONE) != MQTT_OK) ||
        (mqtt_platform_listen(NULL, "1883", &listener) != MQTT_OK)) {
        fprintf(stderr, "the server could not start\n");
        return 1;
    }
    while (!mqtt_platform_stop_requested()) {
        /* 定时器，它也说明循环可以休眠多久。 */
        uint32_t wait_ms = mqtt_server_tick(&server, mqtt_platform_time_ms());

        /* 新连接。 */
        while (mqtt_platform_accept(listener, &handle) == MQTT_OK) {
            if ((mqtt_server_connection_open(&server, mqtt_platform_time_ms(),
                                             &c) == MQTT_OK) &&
                (c < CONNECTIONS)) {
                sockets[c] = handle;
            } else {
                mqtt_platform_close(handle); /* 没有空间再容纳一个 */
            }
        }
        /* 每个打开的连接。 */
        for (c = 0U; c < CONNECTIONS; c++) {
            if (sockets[c] != MQTT_SOCKET_NONE) {
                serve(c);
            }
        }
        mqtt_platform_sleep_ms((wait_ms < ROUND_MS) ? wait_ms : ROUND_MS);
    }
    for (c = 0U; c < CONNECTIONS; c++) {
        if (sockets[c] != MQTT_SOCKET_NONE) {
            close_connection(c);
        }
    }
    mqtt_platform_close(listener);
    return 0;
}
```

api-reference-zh.tex Page 110 November 8, 2026 13:47

5.2 客户端

client_setup.c 为客户端提供内存，client_setup.h 声明它。

client_setup.h

```
/* 示例客户端共用的东西：它们的内存。 */
#ifndef CLIENT_SETUP_H
#define CLIENT_SETUP_H

#include "base_client.h"

/* 给客户端提供内存，并用 on_event 接收所发生的事。 */
mqtt_status client_setup(mqtt_client *client,
                         mqtt_client_event_function on_event);

#endif /* CLIENT_SETUP_H */
```

client_setup.c

```
/* 示例客户端的内存。 */
#include <stdint.h>

#include "base_client.h"
#include "client_setup.h"

#define RECEIVE_BYTES 65536U  /* 它能接收的最大报文 */
#define BLOCK_SIZE    64U
#define BLOCKS        1024U   /* 64 KiB 等待发送的字节 */
#define MESSAGES      64U
#define INCOMING      64U     /* 它的 Receive Maximum */
#define PENDING       8U
#define ALIASES       16U     /* 它的 Topic Alias Maximum */
#define ARENA_UNITS   1024U   /* 64 KiB，用于它暂存的报文 */

static uint8_t receive_bytes[RECEIVE_BYTES];
static uint8_t pool_bytes[BLOCKS * BLOCK_SIZE];
static uint32_t pool_next[BLOCKS];
static mqtt_client_message messages[MESSAGES];
static uint16_t incoming[INCOMING];
static mqtt_client_pending pending[PENDING];
static mqtt_client_alias aliases[ALIASES];
static uint8_t arena_bytes[ARENA_UNITS * MQTT_ARENA_UNIT];
static uint8_t arena_states[ARENA_UNITS];

mqtt_status client_setup(mqtt_client *client,
                         mqtt_client_event_function on_event)
{
    mqtt_client_storage storage;

    storage.on_event = on_event;
    storage.user_tag = 0U;
    storage.receive_bytes = receive_bytes;
    storage.receive_size = RECEIVE_BYTES;
```

api-reference-zh.tex Page 111 November 8, 2026 13:47


```
    storage.pool_bytes = pool_bytes;
    storage.pool_next = pool_next;
    storage.pool_block_size = BLOCK_SIZE;
    storage.pool_block_count = BLOCKS;
    storage.messages = messages;
    storage.message_count = MESSAGES;
    storage.incoming = incoming;
    storage.incoming_count = INCOMING;
    storage.pending = pending;
    storage.pending_count = PENDING;
    storage.aliases = aliases;
    storage.alias_count = ALIASES;
    storage.arena_bytes = arena_bytes;
    storage.arena_states = arena_states;
    storage.arena_units = ARENA_UNITS;
    return mqtt_client_init(client, &storage);
}
```

client_subscribe.c 用 MQTT 5.0 订阅 a/b，打印到达的前三条消息，然后断开连接。

client_subscribe.c

```
/* 订阅 a/b 并打印前三条消息的客户端。 */
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

#include "base_client.h"
#include "base_net.h"
#include "base_platform.h"
#include "client_setup.h"
#include "v5_rc.h"

#define STEP_MS 100U

static mqtt_client client;
static bool accepted = false;   /* 有 CONNACK 接受了连接 */
static bool refused = false;    /* 服务器拒绝了 */
static bool subscribed = false; /* SUBACK 已到达 */
static uint32_t received = 0U;

/* 服务器发来的一切都到这里，在 mqtt_net_client_step 期间。 */
static void on_event(uint32_t tag, const mqtt_client_event *event)
{
    (void)tag;
    switch (event->type) {
    case MQTT_CLIENT_EVENT_CONNACK:
        accepted = (event->connack->reason_code == 0U);
        refused = !accepted;
        break;
    case MQTT_CLIENT_EVENT_SUBACK:
        subscribed = true;
        refused = (event->subscribed->reason_codes.length == 0U) ||
                  (event->subscribed->reason_codes.data[0] >=
```

api-reference-zh.tex Page 112 November 8, 2026 13:47


```
                    MQTT5_RC_FAILURE_MIN);
        break;
    case MQTT_CLIENT_EVENT_MESSAGE:
        printf("%.*s %.*s\n", (int)event->publish->topic_name.length,
               (const char *)event->publish->topic_name.data,
               (int)event->publish->payload.length,
               (const char *)event->publish->payload.data);
        received += 1U;
        break;
    default:
        break;
    }
}

/* 推进连接，直到 *flag 被设置、连接被拒绝，或者连接结束。 */
static bool wait_for(mqtt_net_client *network, const bool *flag)
{
    mqtt_status status = MQTT_OK;

    while (!*flag && !refused && (status == MQTT_OK) &&
           !mqtt_platform_stop_requested()) {
        status = mqtt_net_client_step(network, STEP_MS);
    }
    return *flag && !refused;
}

int main(void)
{
    mqtt_net_client_tls tls;
    mqtt_net_client network;
    mqtt_connect connect;
    mqtt_client_subscription entry;
    uint16_t packet_identifier;
    bool done = false;

    if (client_setup(&client, &on_event) != MQTT_OK) {
        return 1;
    }

    tls.enabled = false;
    tls.insecure = false;
    tls.ca_file = NULL;
    tls.server_name = NULL;
    tls.certificate_file = NULL;
    tls.key_file = NULL;
    tls.key_password = NULL;
    if (mqtt_net_client_open(&network, &client, "localhost", "1883", &tls) !=
        MQTT_OK) {
        return 1;
    }

    mqtt_connect_clear(&connect);
    connect.version = MQTT_VERSION_5;
    connect.clean_start = true;
    connect.keep_alive = 60U;
    connect.client_identifier = mqtt_utf8_from_text("reader");
    if ((mqtt_client_connect(&client, &connect, mqtt_platform_time_ms()) ==
```

api-reference-zh.tex Page 113 November 8, 2026 13:47


```
         MQTT_OK) &&
        wait_for(&network, &accepted)) {
        entry.topic_filter = mqtt_utf8_from_text("a/b");
        entry.options = 1U; /* QoS 1，没有其它选项 */
        if ((mqtt_client_subscribe(&client, &entry, 1U, NULL,
                                   &packet_identifier) == MQTT_OK) &&
            wait_for(&network, &subscribed)) {
            fprintf(stderr, "subscribed\n");
            while ((received < 3U) && !mqtt_platform_stop_requested() &&
                   (mqtt_net_client_step(&network, STEP_MS) == MQTT_OK)) {
            }
            done = (received == 3U);
        }
    }
    if (mqtt_client_disconnect(&client, NULL) == MQTT_OK) {
        while (mqtt_net_client_step(&network, STEP_MS) == MQTT_OK) {
        }
    }
    mqtt_net_client_close(&network);
    return done ? 0 : 1;
}
```

client_publish.c 以 QoS 1 向 a/b 发布三条消息，带一个 Content Type 和一个用户属性，并等待每一条都得到确认。它的 CONNECT 请求一个比连接多存在五分钟的会话。

client_publish.c

```
/* 以 QoS 1 向 a/b 发布三条带属性的消息的客户端。 */
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

#include "base_client.h"
#include "base_net.h"
#include "base_platform.h"
#include "client_setup.h"
#include "v5_prop.h"

#define STEP_MS 100U
#define COUNT   3U

static mqtt_client client;
static bool accepted = false;  /* 有 CONNACK 接受了连接 */
static bool refused = false;   /* 服务器拒绝了 */
static uint32_t answered = 0U; /* 流程已经结束的消息 */

static void on_event(uint32_t tag, const mqtt_client_event *event)
{
    (void)tag;
    if (event->type == MQTT_CLIENT_EVENT_CONNACK) {
        accepted = (event->connack->reason_code == 0U);
        refused = !accepted;
```

api-reference-zh.tex Page 114 November 8, 2026 13:47


```
    } else if (event->type == MQTT_CLIENT_EVENT_PUBLISHED) {
        /* QoS 1 消息的 PUBACK: 0x00，没有人订阅时为 0x10。 */
        fprintf(stderr, "packet %u answered with 0x%02x\n",
                (unsigned)event->response->packet_identifier,
                (unsigned)event->response->reason_code);
        answered += 1U;
    } else {
        /* 其它的在这里都无关紧要。 */
    }
}

int main(void)
{
    static const mqtt5_utf8_pair origin = {{6U, (const uint8_t *)"source"},
                                           {7U, (const uint8_t *)"example"}};
    mqtt_net_client_tls tls;
    mqtt_net_client network;
    mqtt_connect connect;
    mqtt_publish message;
    char text[32];
    uint32_t sent = 0U;
    mqtt_status status = MQTT_OK;

    if (client_setup(&client, &on_event) != MQTT_OK) {
        return 1;
    }
    memset(&tls, 0, sizeof(tls)); /* 不用 TLS */
    if (mqtt_net_client_open(&network, &client, "localhost", "1883", &tls) !=
        MQTT_OK) {
        return 1;
    }

    mqtt_connect_clear(&connect);
    connect.version = MQTT_VERSION_5;
    connect.clean_start = true;
    connect.keep_alive = 60U;
    connect.client_identifier = mqtt_utf8_from_text("writer");
    /* 一个比连接多存在五分钟的会话。 */
    connect.properties.session_expiry_interval = 300U;
    mqtt5_properties_mark(&connect.properties,
                          MQTT5_PROP_SESSION_EXPIRY_INTERVAL);
    status = mqtt_client_connect(&client, &connect, mqtt_platform_time_ms());
    while ((status == MQTT_OK) && !accepted && !refused) {
        status = mqtt_net_client_step(&network, STEP_MS);
    }

    while ((status == MQTT_OK) && accepted && (answered < COUNT) &&
           !mqtt_platform_stop_requested()) {
        if (sent < COUNT) {
            memset(&message, 0, sizeof(message)); /* 没有属性 */
            snprintf(text, sizeof(text), "hello %u", (unsigned)(sent + 1U));
            message.qos = 1U;
            message.topic_name = mqtt_utf8_from_text("a/b");
            message.payload.data = (const uint8_t *)text;
            message.payload.length = (uint32_t)strlen(text);
            message.properties.content_type =
                mqtt_utf8_from_text("text/plain");
```

api-reference-zh.tex Page 115 November 8, 2026 13:47


```
            mqtt5_properties_mark(&message.properties,
                                  MQTT5_PROP_CONTENT_TYPE);
            message.properties.user_pairs = &origin;
            message.properties.user_pair_count = 1U;
            /* 消息会被复制：text 可以立即重用。 */
            if (mqtt_client_publish(&client, &message,
                                    mqtt_platform_time_ms()) == MQTT_OK) {
                sent += 1U;
            }
        }
        status = mqtt_net_client_step(&network, (sent < COUNT) ? 0U : STEP_MS);
    }

    if (mqtt_client_disconnect(&client, NULL) == MQTT_OK) {
        while (mqtt_net_client_step(&network, STEP_MS) == MQTT_OK) {
        }
    }
    mqtt_net_client_close(&network);
    return (answered == COUNT) ? 0 : 1;
}
```

A

全部函数

本文档的函数按名字排列，附有它们的头文件和描述它们的那一节。

表 A.1 全部函数, 按名字排列

函数	头文件	§
mqtt5_properties_clear	v5_properties.h	4.2.1
mqtt5_properties_has	v5_properties.h	4.2.3
mqtt5_properties_mark	v5_properties.h	4.2.2
mqtt5_property_next	v5_properties.h	4.2.4
mqtt5_reason_code	base_packet.h	4.3.3
mqtt_client_assume_session	base_client.h	3.5.2
mqtt_client_auth	base_client.h	3.6.2
mqtt_client_connect	base_client.h	3.6.1
mqtt_client_connection_lost	base_client.h	3.6.5
mqtt_client_disconnect	base_client.h	3.6.3
mqtt_client_init	base_client.h	3.5.1
mqtt_client_publish	base_client.h	3.7.1
mqtt_client_receive	base_client.h	3.9.4
mqtt_client_receive_capacity	base_client.h	3.9.1
mqtt_client_receive_commit	base_client.h	3.9.3
mqtt_client_receive_space	base_client.h	3.9.2
mqtt_client_restore	base_client.h	3.11.1

续页

api-reference-zh.tex Page 118 November 8, 2026 13:47

表 A.1 全部函数, 按名字排列(续)

函数	头文件	§
mqtt_client_should_close	base_client.h	3.6.4
mqtt_client_snapshot_abort	base_client.h	3.11.10
mqtt_client_snapshot_begin	base_client.h	3.11.7
mqtt_client_snapshot_done	base_client.h	3.11.9
mqtt_client_snapshot_due	base_client.h	3.11.6
mqtt_client_snapshot_next	base_client.h	3.11.8
mqtt_client_store_commit	base_client.h	3.11.2
mqtt_client_store_failed	base_client.h	3.11.4
mqtt_client_store_identifier	base_client.h	3.10.4
mqtt_client_store_init	base_client.h	3.10.1
mqtt_client_store_open	base_client.h	3.11.5
mqtt_client_stored	base_client.h	3.11.3
mqtt_client_subscribe	base_client.h	3.7.2
mqtt_client_tick	base_client.h	3.9.7
mqtt_client_transmit_consume	base_client.h	3.9.6
mqtt_client_transmit_view	base_client.h	3.9.5
mqtt_client_unsubscribe	base_client.h	3.7.3
mqtt_connect_clear	base_packet.h	4.3.2
mqtt_library_compatible	base_library.h	4.7.1
mqtt_library_page_size	base_library.h	4.7.3
mqtt_library_version	base_library.h	4.7.2
mqtt_net_client_close	base_net.h	3.8.3
mqtt_net_client_open	base_net.h	3.8.1
mqtt_net_client_restore	base_net.h	3.10.3
mqtt_net_client_step	base_net.h	3.8.2
mqtt_net_client_store_config	base_net.h	3.10.2

续页

api-reference-zh.tex Page 119 November 8, 2026 13:47

表 A.1 全部函数, 按名字排列(续)

函数	头文件	§
mqtt_net_server_counters	base_net.h	2.6.2
mqtt_net_server_restore	base_net.h	2.9.2
mqtt_net_server_run	base_net.h	2.6.1
mqtt_net_server_store_config	base_net.h	2.9.1
mqtt_platform_random_bytes	base_platform.h	4.6.2
mqtt_platform_request_stop	base_platform.h	4.6.4
mqtt_platform_stop_requested	base_platform.h	4.6.3
mqtt_platform_store_close	base_platform.h	4.5.3
mqtt_platform_store_open	base_platform.h	4.5.2
mqtt_platform_time_ms	base_platform.h	4.6.1
mqtt_reason_clear	base_packet.h	4.3.1
mqtt_server_accepted	base_server.h	2.7.14
mqtt_server_config_default	base_server.h	2.5.1
mqtt_server_connection_closed	base_server.h	2.7.2
mqtt_server_connection_open	base_server.h	2.7.1
mqtt_server_disconnect	base_server.h	2.7.17
mqtt_server_idle	base_server.h	2.7.16
mqtt_server_init	base_server.h	2.5.3
mqtt_server_keep_alive	base_server.h	2.7.15
mqtt_server_meta_words	base_server.h	2.5.2
mqtt_server_next_pending	base_server.h	2.7.11
mqtt_server_pool_audit	base_server.h	2.11.1
mqtt_server_publish	base_server.h	2.8.1
mqtt_server_receive	base_server.h	2.7.7
mqtt_server_receive_capacity	base_server.h	2.7.4
mqtt_server_receive_commit	base_server.h	2.7.6

续页

api-reference-zh.tex Page 120 November 8, 2026 13:47

表 A.1 全部函数, 按名字排列(续)

函数	头文件	§
mqtt_server_receive_space	base_server.h	2.7.5
mqtt_server_receive_wanted	base_server.h	2.7.3
mqtt_server_restore	base_server.h	2.10.1
mqtt_server_retained_next	base_server.h	2.8.2
mqtt_server_retained_restore	base_server.h	2.8.3
mqtt_server_should_close	base_server.h	2.7.10
mqtt_server_snapshot_abort	base_server.h	2.10.10
mqtt_server_snapshot_begin	base_server.h	2.10.7
mqtt_server_snapshot_done	base_server.h	2.10.9
mqtt_server_snapshot_due	base_server.h	2.10.6
mqtt_server_snapshot_next	base_server.h	2.10.8
mqtt_server_store_commit	base_server.h	2.10.2
mqtt_server_store_failed	base_server.h	2.10.4
mqtt_server_store_open	base_server.h	2.10.5
mqtt_server_stored	base_server.h	2.10.3
mqtt_server_take_activity	base_server.h	2.7.13
mqtt_server_tick	base_server.h	2.7.12
mqtt_server_transmit_consume	base_server.h	2.7.9
mqtt_server_transmit_view	base_server.h	2.7.8
mqtt_store_fault_name	base_store.h	4.5.1
mqtt_tls_available	base_tls.h	4.4.1
mqtt_utf8_from_text	base_wire.h	4.1.1